IMPLEMENTATION OF INVENTORY AND SALES MANAGEMENT SYSTEM FOR A RETAIL STORE

BY

RAMINEDI LAXMI HARIKA

24BCE5404

A project report submitted

for the course of

BCSE302L - Database Systems

in

B. Tech. COMPUTER SCIENCE AND ENGINEERING



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

April 2026

Chapter	Title	Page No.
1	Abstract	3
2	Introduction	4
3	Existing Work	5
4	Proposed Work / System	6
5	System design / Architecture of proposed work	8
6	Technology Stack	16
7	Working modules with appropriate diagram and explanation	18
8	Screenshots of Output	21
9	Conclusion	37
10	References	38
11	Complete Implementation	39

ABSTRACT

Inventory management is an essential component in the success of retail businesses. Lack of control in managing inventory, suppliers, and sales can lead to financial losses, excess inventory, and dissatisfied customers. Conventional approaches, such as manual records or using spreadsheets, are unable to offer accurate information in real-time, which is essential in managing the ever-changing environment of retail businesses.

This project introduces **InventoryPro**, which is an efficient, full-stack, web-based Inventory and Sales Management System developed with the aim of eliminating the limitations of conventional retail management practices. It is developed using a well-structured normalized database schema, which is highly efficient in maintaining data integrity in all operations.

InventoryPro offers four unique role-authenticated portals: an **Admin Portal** for overall management and analytics, an **Employee/Cashier Portal** for executing sales transactions and tracking individual performance, a **Supplier Portal** for processing purchase orders, tracking deliveries, and payments, and a **Customer Portal** for self-service order tracking and purchase history.

The system's backend is implemented using **Node.js** with the **Express.js** framework, connecting to a MySQL relational database through a connection pool managed by the **`mysql2`** library. The frontend is built using **React.js** with **Vite**, employing **TanStack Query** for server-state synchronisation and **Framer Motion** for fluid micro-animations. **JSON Web Tokens** (JWT) provide role-based authentication and route protection across all portals.

The system has all the essential functionalities, such as **product catalog** management, **inventory** management, **purchase order** management, point of sale billing, and financial reporting. The system also has some essential features, such as real-time admin dashboard showing essential KPIs, revenue chart with month-wise filtering, and alerts for low stock and expired products. The system also has an **employee performance** dashboard showing cashier efficiency and a **supplier management** system showing purchase order and delivery status. Overall, the project is an example of the effective application of database design principles such as normalization, constraint management, and query optimization in real-world scenarios.

INTRODUCTION

Small-to-medium retail businesses have issues such as stock variations, improper supplier coordination, absence of expiry dates, insufficient employee performance tracking, and lack of real-time financial information. Also, the traditional systems do not offer proper access control for different users.

To overcome these challenges, this project proposes the development of an "**Inventory and Sales Management System**" using web technology, which will increase the efficiency, accuracy, and transparency of the retail business processes. The proposed system will be based on normalized relational databases, and provisions for different types of users, i.e., administrators, employees, suppliers, and customers, will be incorporated. This system is also having some of the important modules of an inventory and sales management system, i.e., real-time inventory management, low stock and expiry date alerts, Point of Sale billing, GST and HSN code, supplier order management, and analytical reports.The system will also include real-time business monitoring using dashboards that show KPIs and revenue graphs.

It also includes other basic retail operations that include product management, inventory management, supplier management, employee management, customer management, billing, among others. However, all these operations are carried out in one system. Moreover, there is also an audit trail for any changes in the inventory, which makes it transparent. The system also provides effective data retrieval and processing, which makes it more efficient in terms of user experience. Therefore, it provides a user-friendly solution that makes it easy to manage all retail operations.

EXISTING WORK

Currently, there are several inventory management systems that could be used, including **Zoho Inventory, Tally**, **inFlow**, and **Odoo**. Each of these systems offers valuable features that could be used in the business, including tracking, order management, accounting, among others. However, Zoho Inventory is not fully integrated with POS systems or user portals. Tally is not user-friendly since it is not accessible on the web. inFlow is user-friendly but cannot be used for complex features such as real-time alerts or employee performance tracking. Odoo is highly flexible but is technically complex.

Despite these advantages, these systems also have limitations like the absence of real-time notifications, absence of multi-role access control, limitations in terms of customizations, and cost and complexity for small businesses. Another limitation is that these systems don't support open relational database structures. There are also limitations in terms of expiry tracking, supplier self-service, and audit trails.

The limitations identified above have clearly pointed out the research gap in developing a lightweight, web-based system with all the necessary functionalities integrated into a single platform. There is a research gap in developing a system with role-based access control, real-time stock and expiry date alerts, integrated POS with GST, and reporting facilities. The proposed system fills the gap by developing a lightweight, user-friendly, scalable system for small to medium-sized retail businesses.

PROPOSED WORK/ SYSTEM

InventoryPro is a web-based, multi-role inventory and sales management system built on a client-server architecture. The system offers four unique user interfaces with JWT authentication and role-based access control.

Admin Portal: The Admin Portal serves as the central hub of InventoryPro, providing comprehensive control over all operations. The Admin Portal enables administrators to do CRUD operations (Create, Read, Update, and Delete) on products, employees, suppliers, warehouses, and customer data, ensuring that the data is accurate and up-to-date in the system. The Admin Portal also enables administrators to generate reports and analyze data, such as revenue graphs, inventory reports, and employee performance reports, which are essential in making business decisions. The system also enables real-time notifications, ensuring that critical operations are performed in an efficient and transparent manner.

Employee (Cashier) Portal: The Employee Portal is intended for smooth Point of Sale (POS) operations. The cashiers can efficiently process sales, considering the availability of the inventory in real-time to avoid overselling. The Employee Portal also includes a personal employee performance chart, which displays the number of sales made on a daily basis, total sales made in their lifetime, revenue generated, progress of their shifts, and accuracy of payment. This enables them to efficiently monitor their performance.Role-based access ensures that cashiers can only access functions relevant to their job, maintaining data security while offering all necessary tools for day-to-day operations.

Supplier Portal: The Supplier Portal empowers suppliers to actively manage their interactions with the business. Suppliers are able to track their purchase orders, manage their deliveries, and keep track of their payment history. Suppliers are also able to manage their profiles, which enables them to modify their company details and contact information. One of the most innovative features is the ability of the suppliers to manage their deliveries. Suppliers are able to input their shipping modes, purchase agents, and delivery status.

Customer Portal: The Customer Portal improves the user experience by offering customers a way to access order history, payments, and profile information. Customers are able to keep track of orders and manage information such as shipping addresses and payment information. The Customer Portal promotes a transparent environment and allows customers to manage orders on their own, resulting in a better user experience.

Key Innovations of InventoryPro

- **Real-Time Expiry Alerts:** Tracks expired products and items expiring within 30 days, displayed as interactive badges in the Admin Dashboard with detailed modals for inventory location and expiry dates.

- **Configurable Reorder Levels:** Custom reorder levels for every product in every inventory, triggering alerts for products going below that reorder level.

- **Employee Performance Dashboard:** Cashiers receive information on total transactions, revenue, daily targets, and accuracy of payments.

- **Revenue Trend Filtering:** Admins receive information on sales trends for specific time periods, with dynamic SQL queries for grouped daily data for specific time periods.

- **Advanced Search and Filtering:** Users receive quick search results for products, orders, and transactions through various filters like category, supplier, expiry date, and warehouse location.

- **HSN/GST Compliance:** Products are assigned HSN tax codes, and tax is automatically calculated for each item in the POS module.

- **Mandatory-Reason Stock Auditing:** Manually updating stock levels (IN/OUT) requires a reason and is tagged with employee ID for a secure and unalterable audit.

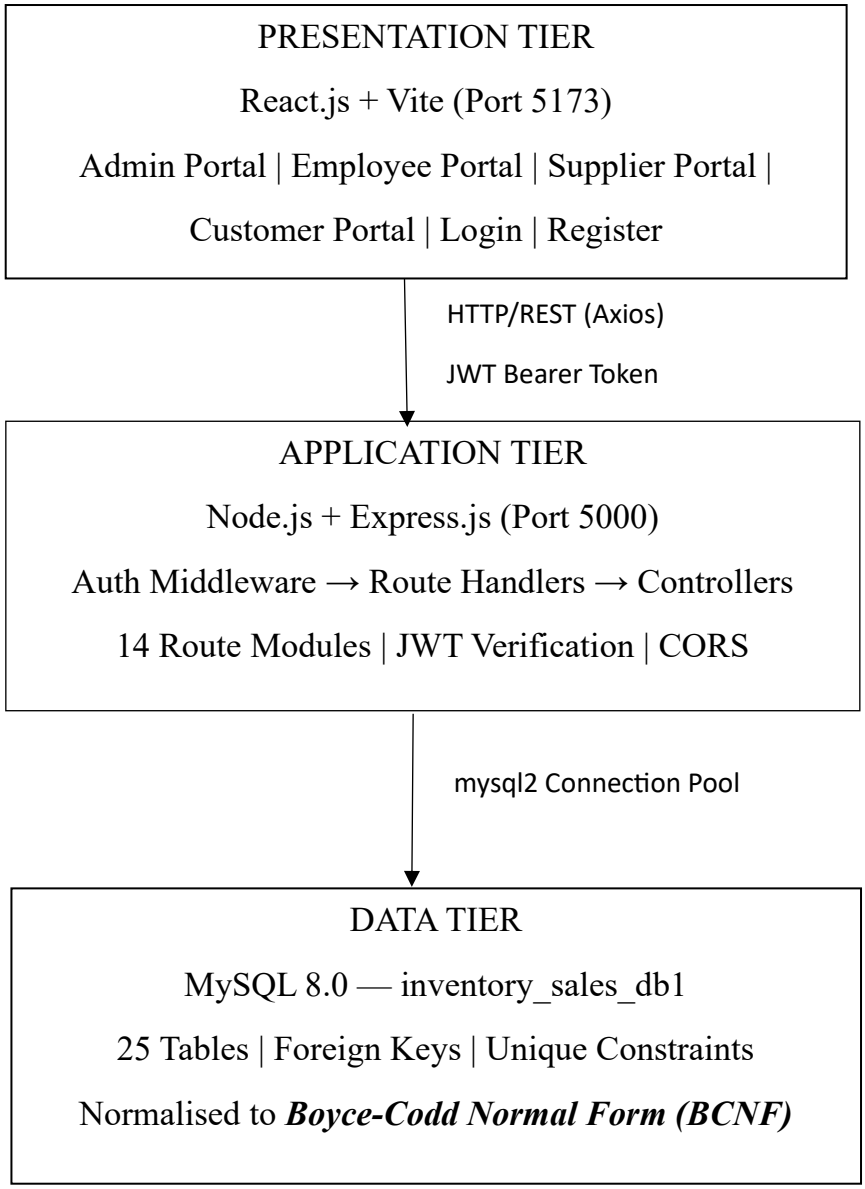
- **Supplier-Driven Delivery Management:** Suppliers log in and manage shipping methods, purchase agent information, and delivery information through their own supplier portal.

- **Scalable Architecture:** Node.js connection pooling, optimized SQL queries, and the use of React Query for auto-refreshing data guarantee smooth performance even with multiple users.

Overall, InventoryPro provides a **secure, efficient, and feature-rich inventory and sales management solution**, combining real-time insights, multi-portal accessibility, role-specific dashboards, and enhanced operational control to outperform existing systems.

SYSTEM DESIGN/ARCHITECTURE

InventoryPro follows a three-tier architecture:



The database consists of 25 tables organised into 7 functional domains:

Domain 1: Core Master Data

- **Category** - Product classification with Categorgy_id as primary key
- **Hsn_Tax** - HSN code to GST rate mapping for tax compliance.

Domain 2: Users and Stakeholders

- **Employee_Address** - Normalised address table keyed by pincode (BCNF).
- **Employee** - Staff records with role field ('Admin'/'Cashier'), FK to address.
- **Customer** - Consumer records with unique email constraint.

Domain 3: Suppliers

- **Supplier_Address** - Normalised supplier address by pincode.
- **Supplier_Bank_Details** - Banking information separated with bank_account_number as primary key
- **Supplier** - Supplier entity with unique bank account constraint.
- **Supplier_Contact** - Multi-valued phone contacts (1:N, composite PK).

Domain 4: Products

- **Product** - Core product catalogue with barcode (unique), FK to category, supplier, and HSN code.
- **Product_Description** - Separated description in a 1:1 extension table to avoid wide rows.

Domain 5: Warehouse and Inventory

- **Warehouse_Address** - Normalised warehouse location.
- **Warehouse** - Warehouse entity with manager and pincode.
- **Inventory** - Live stock levels (stock_in, stock_out, current_stock, expiry_date) per product.
- **Product_Reorder** - Configurable reorder threshold per product (composite PK).
- **Stock_Adjustment** - Full audit log of stock changes with employee attribution.

Domain 6: Purchase Orders

- **Purchase_Order_Status** - Valid status values (reference/lookup table).
- **Purchase_Order** - PO header with supplier, dates, total, and status FK.
- **Purchase_Order_Detail** - PO line items (composite PK: po_id + product_id).
- **Delivery_Details** - Shipping agent and mode per PO.

Domain 7: Sales / POS

- **Sales_Transaction_Rates** - Tax category to rate mapping.
- **Sales_Transaction_Fees** - Payment method to processing fee mapping.
- **Sales_Transaction** - Bill header with customer, employee, datetime, total, and payment type.
- **Sales_Transaction_Details** - Bill line items with tax computation (composite PK).
- **Payment** - Payment record linked to transaction.

API Architecture

The backend exposes 14 route modules mounted on the Express application:

/api/auth — Login and registration (**Public Access**)

/api/categories — Product categories (**Admin Only**)

/api/products — Product Management (**Admin**) / Product Inventory View (**Cashier**)

/api/suppliers — Supplier database and bank details (**Admin Only**)

/api/inventory — Stock adjustments & auditing (**Admin**) / Stock Levels (**Cashier**)

/api/customers — Customer profile management (**Admin & Cashier**)

/api/purchases — Purchase orders and supplier restocking (**Admin Only**)

/api/sales — POS transactions and billing interface (**Admin & Cashier**)

/api/employees — Staff management and role assignments (**Admin Only**)

/api/reports — **Inventory Pro** Analytics (**Admin**) / Performance Metrics (**Cashier**)

/api/warehouses — Warehouse and regional location management (**Admin Only**)

/api/tax-rates — HSN compliance and GST rate management (**Admin Only**)

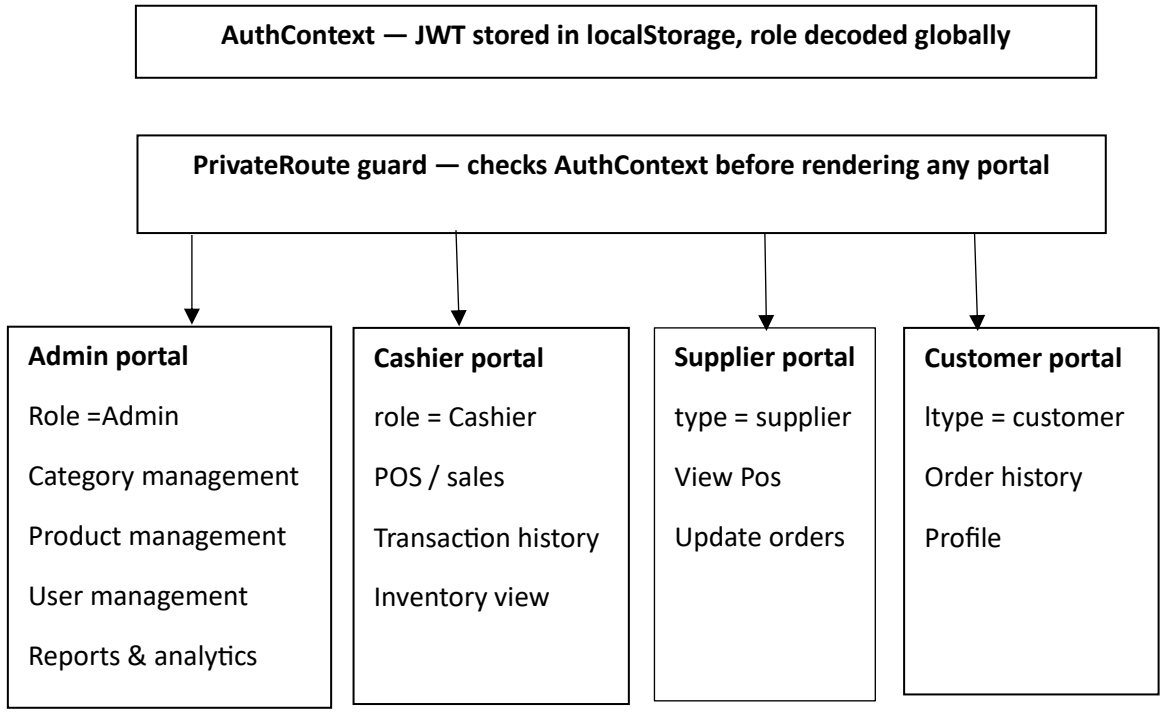
/api/supplier — Supplier self-service portal (**Supplier Access**)

/api/customer — Customer self-service order tracking (**Customer Access**)

Authentication Flow

1. **Submission:** The client sends a POST request with JSON credentials via an encrypted HTTPS connection.
2. **Verification:** The server compares the provided password against the bcrypt-salted hash in the database.
3. **Tokenization:** A stateless JWT is digitally signed by the server, embedding the user's ID, role, and type.
4. **Storage:** The React client persists the token in localStorage and updates the global AuthContext state.
5. **Interception:** Every subsequent API request includes the token in the Authorization: Bearer header.
6. **Validation:** The verifyToken middleware decodes the JWT to ensure it is authentic and not expired.
7. **Authorization:** The authorizeRoles middleware checks if the decoded role has permission to access the route.
8. **Resolution:** The Controller executes the raw SQL query and returns the final JSON data to the UI.

Access control & route guard structure:



Frontend Architecture (Client):

```
client/
└── src/
    ├── main.jsx              # Entry point
    ├── App.jsx               # Routing & role-based access
    ├── index.css             # Global styles (Tailwind)
    ├── App.css               # Custom styles & animations
    │
    ├── context/
    │   └── AuthContext.jsx     # Authentication & session state
    │
    ├── layouts/
    │   ├── DashboardLayout.jsx # Admin/Employee layout
    │   ├── SupplierLayout.jsx  # Supplier portal layout
    │   └── CustomerLayout.jsx  # Customer portal layout
    │
    ├── pages/
    │   ├── Login.jsx
    │   ├── Register.jsx
    │   │
    │   ├── admin/
    │   │   ├── Dashboard.jsx
    │   │   ├── Products.jsx
    │   │   ├── Inventory.jsx
    │   │   ├── Employees.jsx
    │   │   ├── Warehouse.jsx
    │   │   ├── Reports.jsx
    │   │   └── Categories.jsx
    │   │
    │   ├── employee/
```

```
│   │   ├── EmployeeDashboard.jsx
│   │   ├── POS.jsx
│   │   ├── Customers.jsx
│   │   ├── SalesTransaction.jsx
│   │   └── EmployeeProfile.jsx
│   │
│   ├── supplier/
│   │   ├── SupplierDashboard.jsx
│   │   ├── SupplierOrders.jsx
│   │   ├── SupplierDelivery.jsx
│   │   ├── SupplierPayments.jsx
│   │   └── SupplierProfile.jsx
│   │
│   └── customer/
│       ├── CustomerShop.jsx
│       ├── CustomerOrders.jsx
│       └── CustomerProfile.jsx
```

Backend Architecture (Server):

```
server/
├── index.js                # Server entry point
├── .env                    # Environment variables
├── init_db.js              # Database setup script
├── seed_all_products.js    # Data seeding
│
└── src/
    ├── config/
    │   └── db.js           # MySQL connection pool
    │
    ├── middleware/
```

```
│   └── authMiddleware.js   # JWT & RBAC
│
├── routes/
│   ├── authRoutes.js
│   ├── productRoutes.js
│   ├── inventoryRoutes.js
│   ├── salesRoutes.js
│   ├── employeeRoutes.js
│   ├── supplierRoutes.js
│   ├── warehouseRoutes.js
│   ├── reportRoutes.js
│   ├── categoryRoutes.js
│   ├── customerRoutes.js
│   ├── taxRatesRoutes.js
│   ├── customerPortalRoutes.js
│   └── supplierPortalRoutes.js
│
├── controllers/
│   ├── authController.js
│   ├── productController.js
│   ├── inventoryController.js
│   ├── salesController.js
│   ├── employeeController.js
│   ├── supplierController.js
│   ├── warehouseController.js
│   ├── reportController.js
│   ├── categoryController.js
│   ├── customerController.js
│   └── taxRatesController.js
│
```

```
├── models/              # SQL Schema Files
│   ├── 01_core_master.sql
│   ├── 02_users.sql
│   ├── 03_suppliers.sql
│   ├── 04_products.sql
│   ├── 05_warehouse_inventory.sql
│   ├── 06_purchase_orders.sql
│   └── 07_sales_pos.sql
│
└── utils/
    └── helper functions    # Formatting, calculations
```

TECHNOLOGY STACK

Frontend Technologies

React.js (v18) with Vite : React.js is a declarative, component-based JavaScript library for building user interfaces. Vite provides an ultra-fast development server with Hot Module Replacement (HMR), enabling instant reflection of code changes during development. React's virtual DOM diffing algorithm ensures minimal re-rendering for optimal performance**.**

TanStack Query (React Query v5) : TanStack Query handles all server-state management — fetching, caching, synchronising, and invalidating data from the REST API. It automatically re-fetches data when a mutation succeeds, ensuring the UI always reflects the current database state without manual refresh logic.

Framer Motion : Framer Motion is a production-ready animation library for React. It is used extensively in InventoryPro for staggered card entry animations, modal transitions with `AnimatePresence`, and interactive button press effects (`whileHover`, `whileTap`).

Chart.js with react-chartjs-2 : Chart.js renders the revenue velocity trend chart in the Admin Dashboard. The integration uses a linear gradient fill beneath the line chart and custom dark-themed tooltips for a premium visual experience.

Lucide React : Lucide provides a consistent, lightweight SVG icon library used across all components for visual indicators — alert triangles, package icons, calendar icons, search icons, and more.

React Router DOM (v6): React Router provides declarative, client-side routing with support for nested routes and `<Navigate>` components for role-based redirect logic at login.

Tailwind CSS : Tailwind CSS provides utility-first CSS classes used for all layout, typography, spacing, colour, shadow, and animation styling across the application, enabling rapid, consistent design implementation.

Backend Technologies

Node.js : Node.js is a JavaScript runtime built on the V8 engine. Its non-blocking, event-driven I/O model makes it highly suitable for handling concurrent API requests efficiently, particularly for a multi-portal system with simultaneous active sessions.

Express.js : Express.js is a minimal, unopinionated web framework for Node.js. It provides robust routing, middleware chaining, and JSON body parsing capabilities that form the backbone of the InventoryPro REST API.

mysql2 : The ‘mysql2’ package provides a high-performance MySQL client for Node.js. InventoryPro uses it with a connection pool (configured in `database.js`) that maintains a set of reusable database connections, reducing the overhead of establishing a new connection per request.

JSON Web Token (jsonwebtoken) : JWTs are used for stateless authentication. Upon login, the server signs a token embedding the user's ID, type (employee/supplier/customer), and role (Admin/Cashier). This token is verified on every protected API request without requiring a database lookup for the session.

Bcryptjs : All passwords are hashed using bcrypt before storage and verified on login, ensuring no plaintext credentials are stored in the database.

Database

MySQL 8.0 : MySQL is the chosen Relational Database Management System due to its maturity, widespread industry adoption, strong support for foreign key constraints with InnoDB engine, and excellent performance with the 'mysql2' Node.js driver. The database is named 'inventory_sales_db' and contains 22 tables organised into 7 normalised schema groups.

Development Tools

Visual Studio Code - Primary code editor

Postman - API endpoint testing

MySQL Workbench - Database schema visualisation and query testing

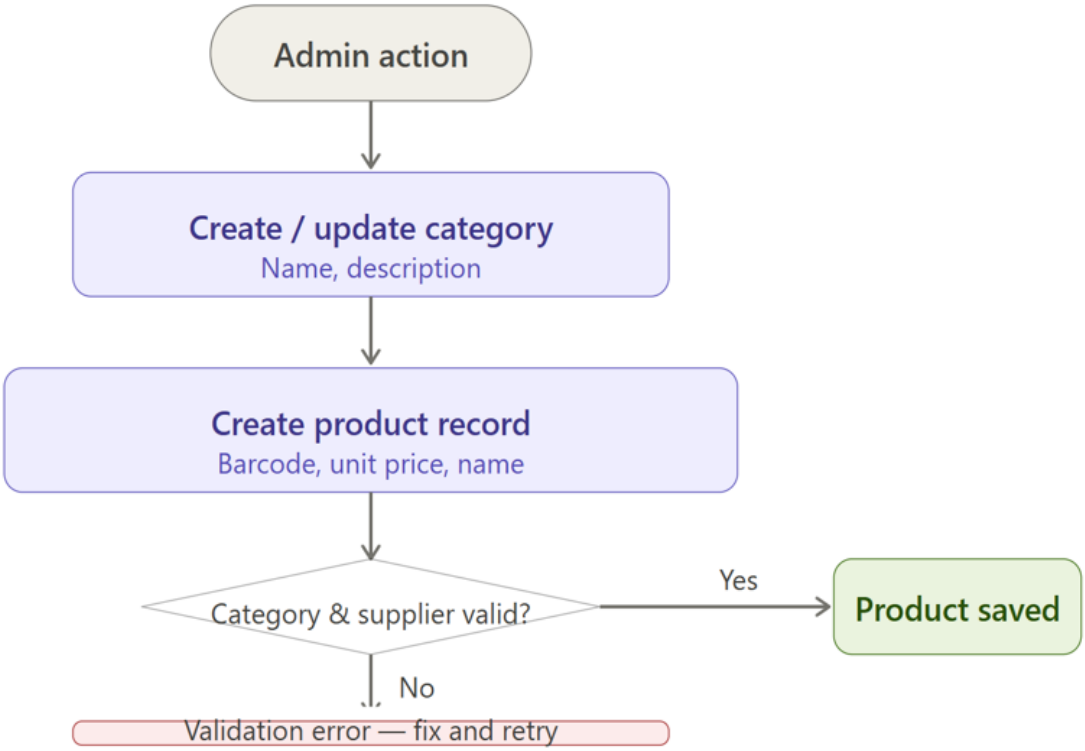
Npm - Package management

WORKING MODULES

Each module is designed to interact with the BCNF-normalized MySQL 8.0 database via a secure Express.js API.

Module 1 - Category & Product Management

This module serves as the base of the system, as there can be no product without linking it to a category and a supplier. It controls data related to products such as the barcode, name, and unit price, which are referenced by other dependent modules such as the inventory, purchase, and sales modules. This ensures data integrity.

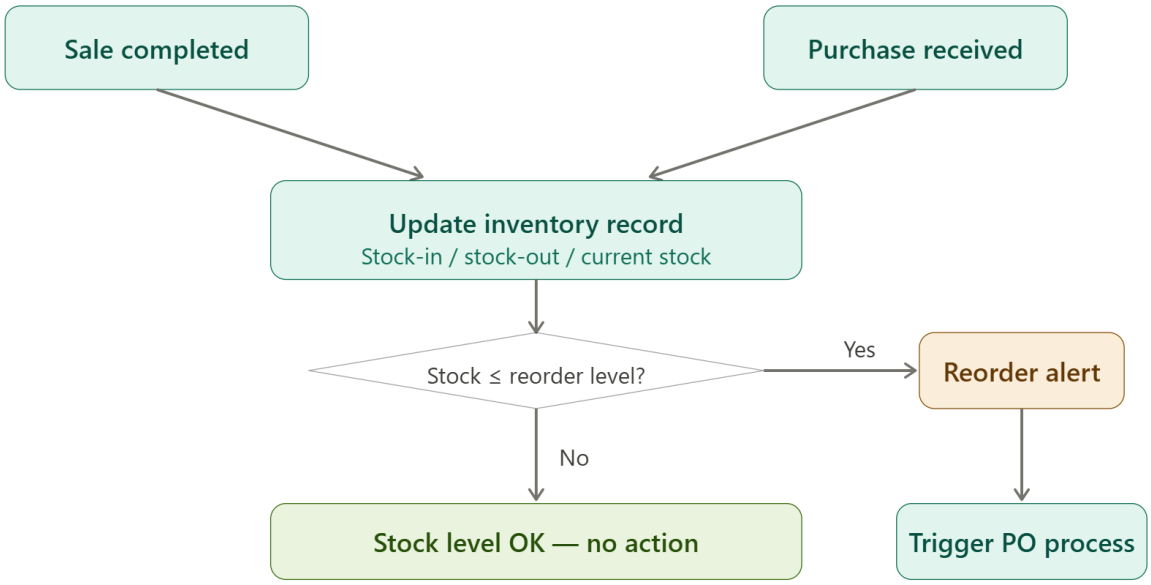


Module 2 — Supplier Management

This module independently maintains all supplier-related master data. It can accommodate multiple contact numbers for each supplier. It can store essential supplier information like addresses and banking details. Supplier master data is used by the Product Management module for assigning products and the Purchase Order module for purchase order activities.

Module 3 - Inventory Management

The Inventory module is a one-to-one relationship with products. This implies that every product will have only one inventory. The information recorded includes stock in, stock out, and stock level. The module will update the stock after every sales and purchase transaction. The module will also send a reorder alert when the stock level falls below a certain level.

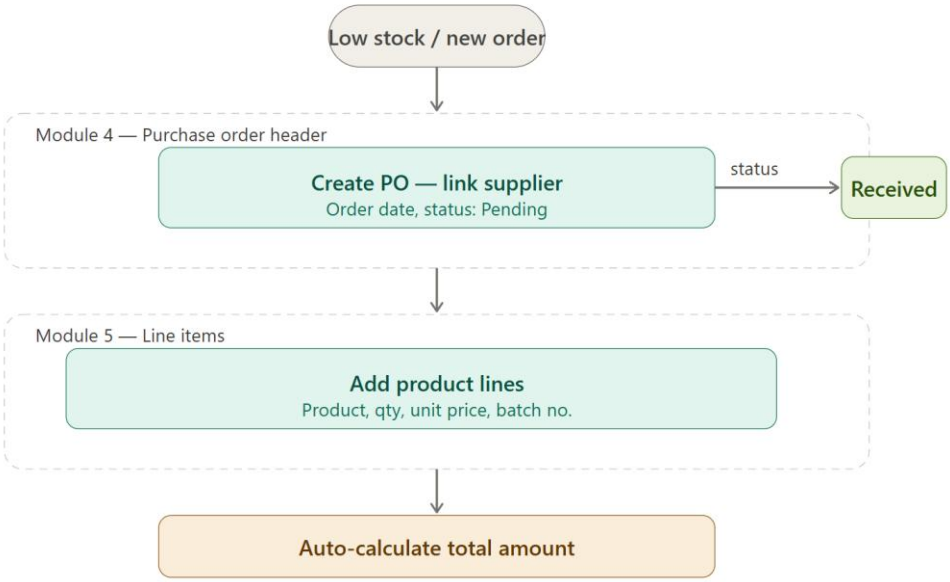


Module 4 — Purchase Order Management

This module is responsible for handling purchase order creation and management. Each purchase order represents a header record associated with a supplier, with properties such as order date and status (e.g., Pending, Processing, Received), and acts as a parent record to a collection of associated purchase order detail records.

Module 5 — Purchase Order Details

This table resolves the many-to-many relationship between the product and purchase order. It consists of one record for each line item in a purchase order, holding specific information about the product. The total value of a purchase order can be automatically calculated based on the line items.



Module 6 — Customer Management

The Customer module stores customer information independently of transactions. Customers can be registered in the system before making any purchases. During sales transactions, customer records are either retrieved or newly created, ensuring seamless customer management and tracking.

Module 7 — Sales Transaction Management

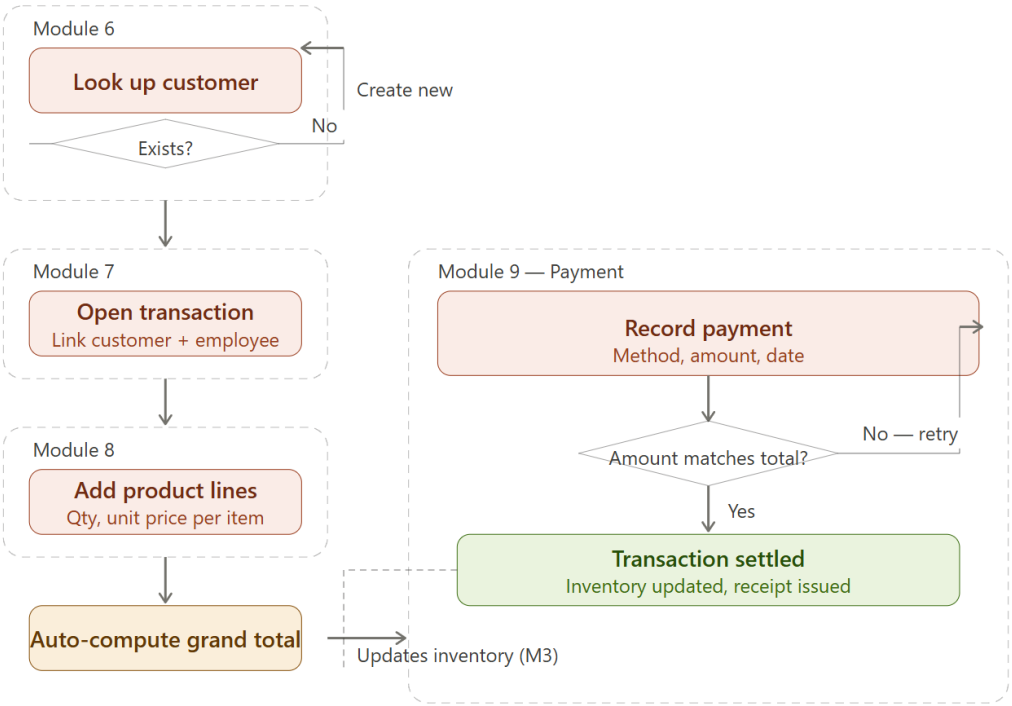
This module serves as the header for all sales transactions. It links each transaction to a specific customer and employee (cashier), in addition to the date and time of the sale. The grand total of each transaction is not manually entered but is automatically calculated on the basis of the related transaction detail

Module 8 — Sales Transaction Details

This module stores information on the individual items of a sales transaction. Each record in this module has information on the item, the quantity of the item, and the price per item. It has a strict dependency on the Sales Transaction module. It cannot exist independently. It plays a vital role in calculating the total amount of the transaction.

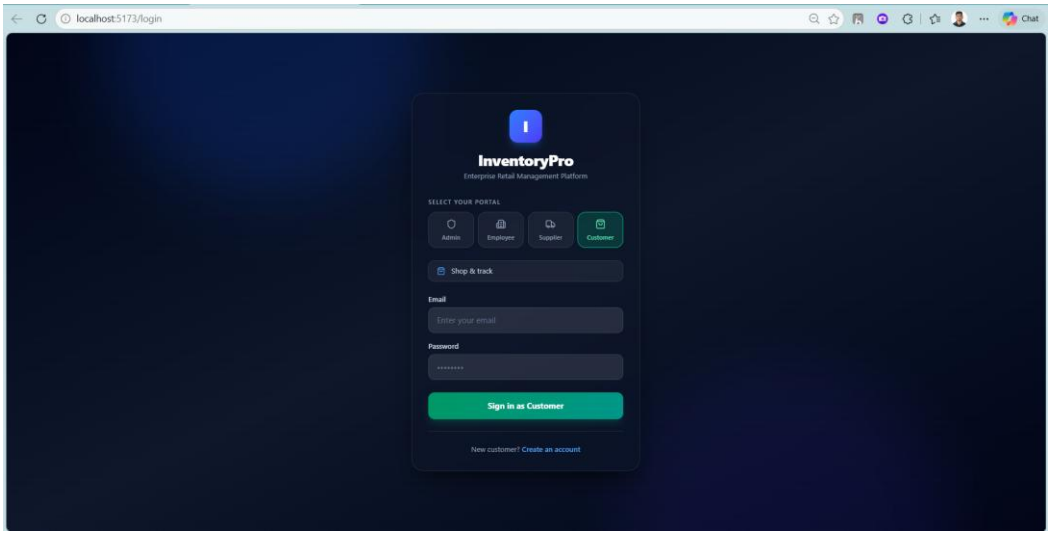
Module 9 — Payment Management

The payment module is responsible for handling the payment procedure for each sales transaction, where a one-to-one relationship is maintained with each transaction. It checks if the payment amount matches the grand total, records the payment mode and date, and marks the transaction as complete. After a successful payment, it initiates stock level reductions accordingly.

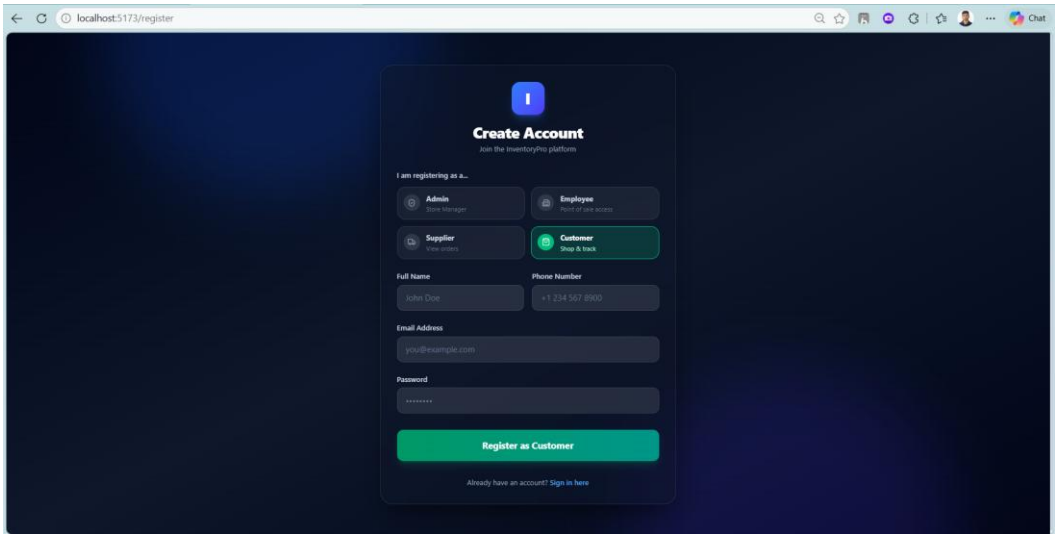


SCREENSHORTS OF THE OUTPUT

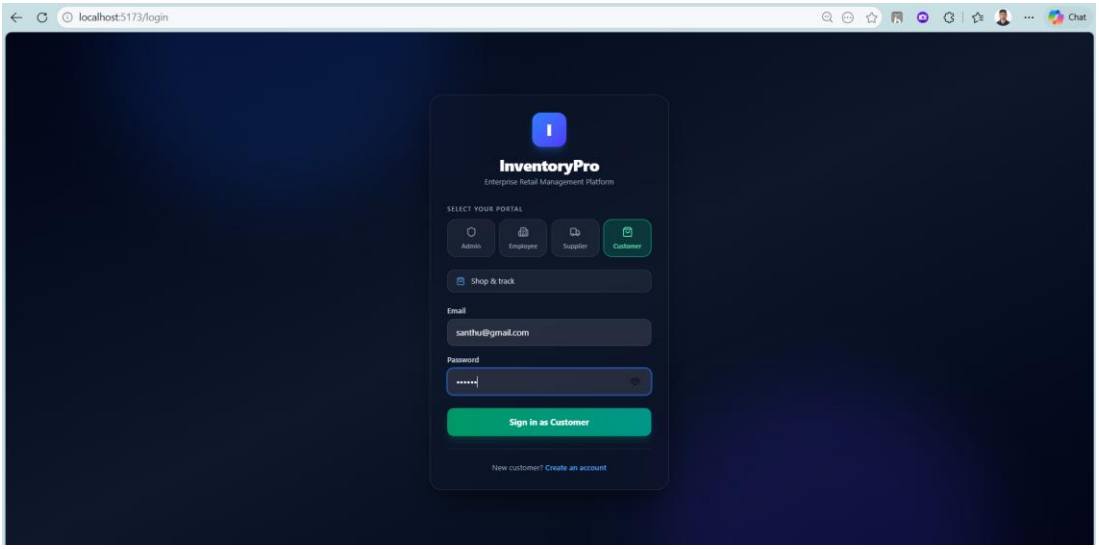
Login page:



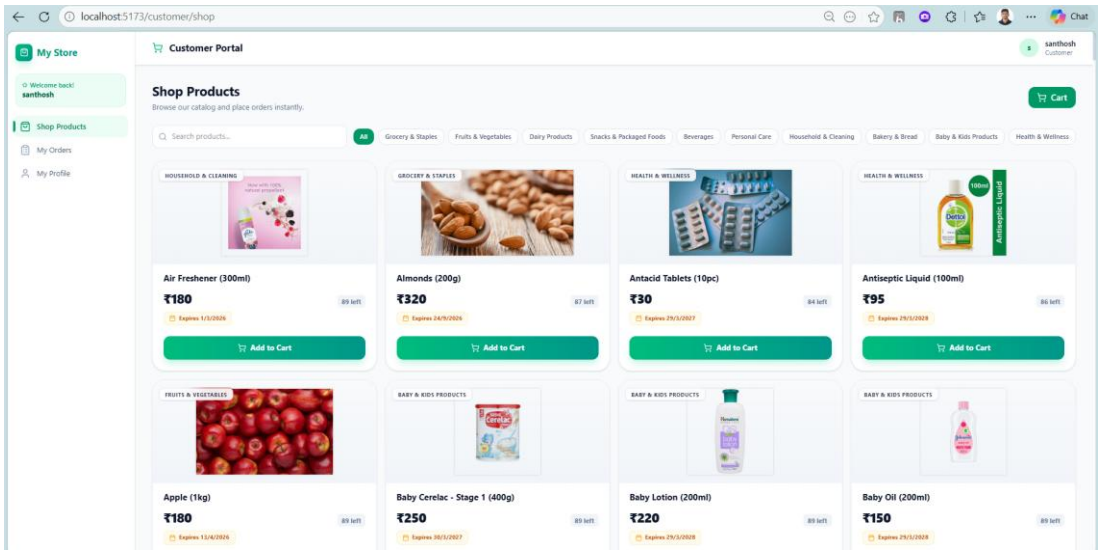
Signup page:



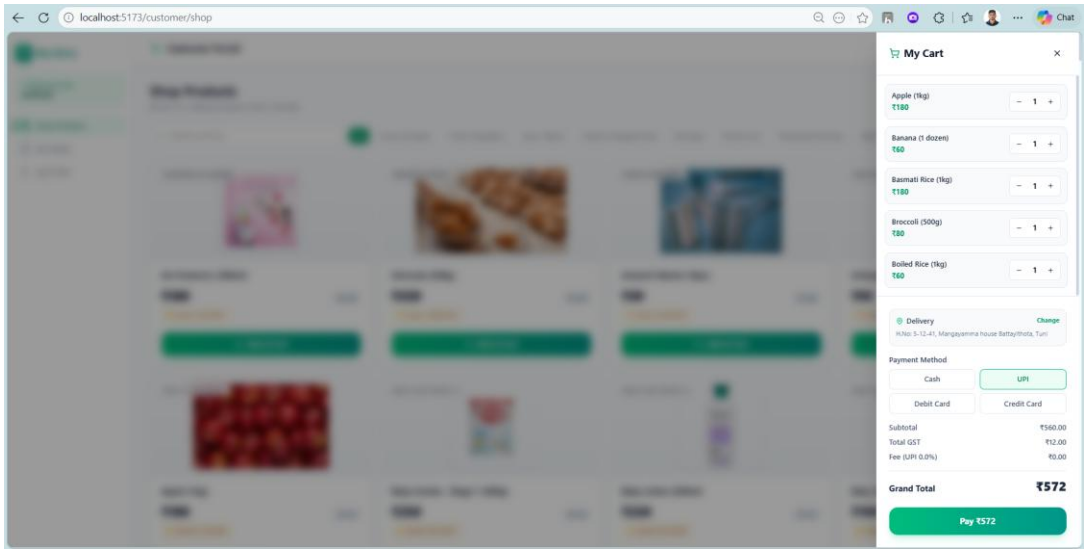
Customer Portal



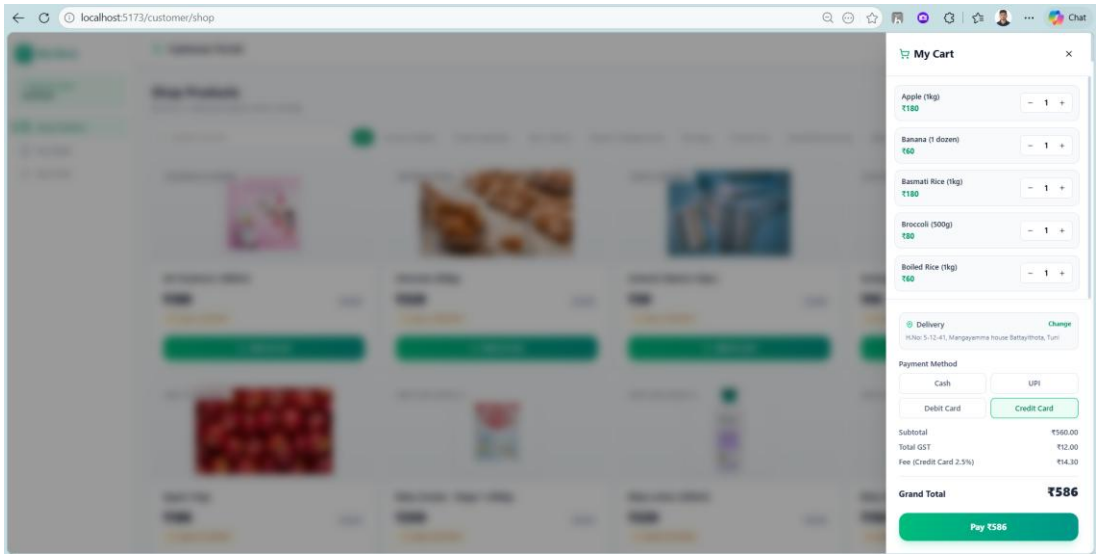
Customer Portal:



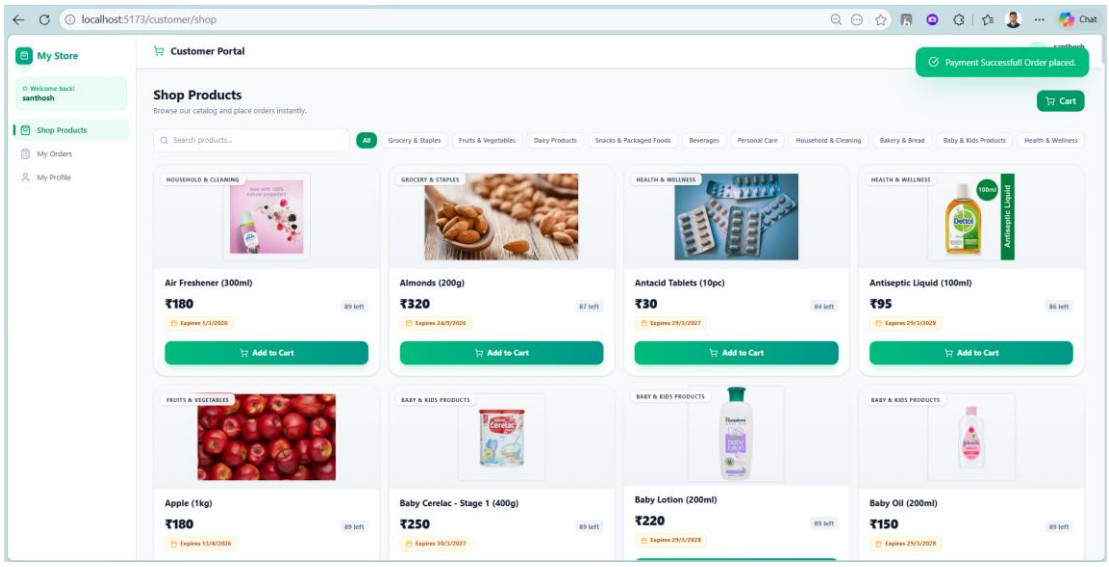
Add to Cart:



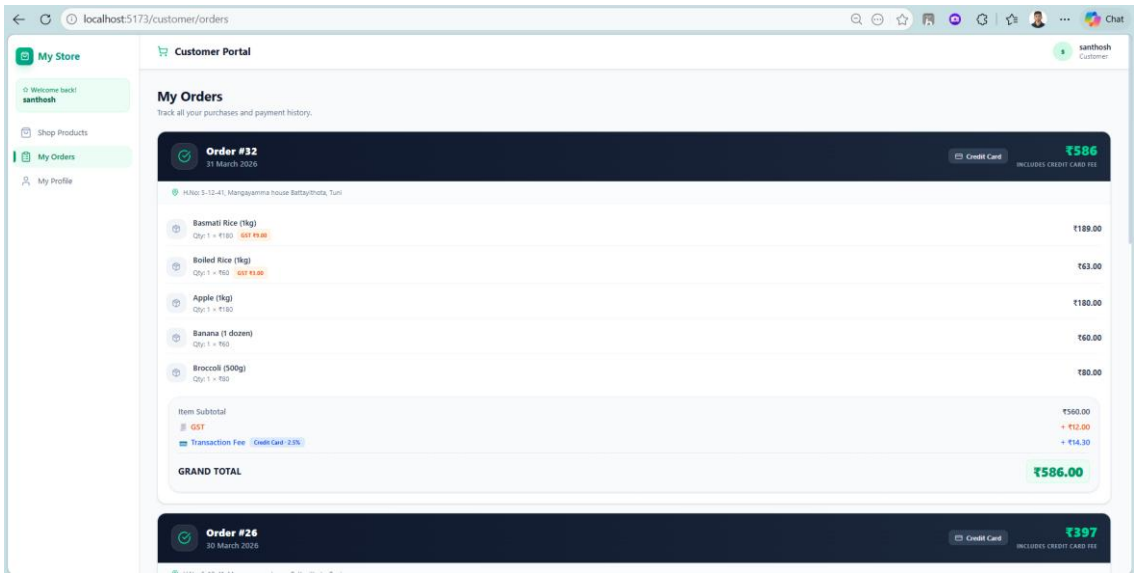
Automatic Gst and Transaction Fee calculation:



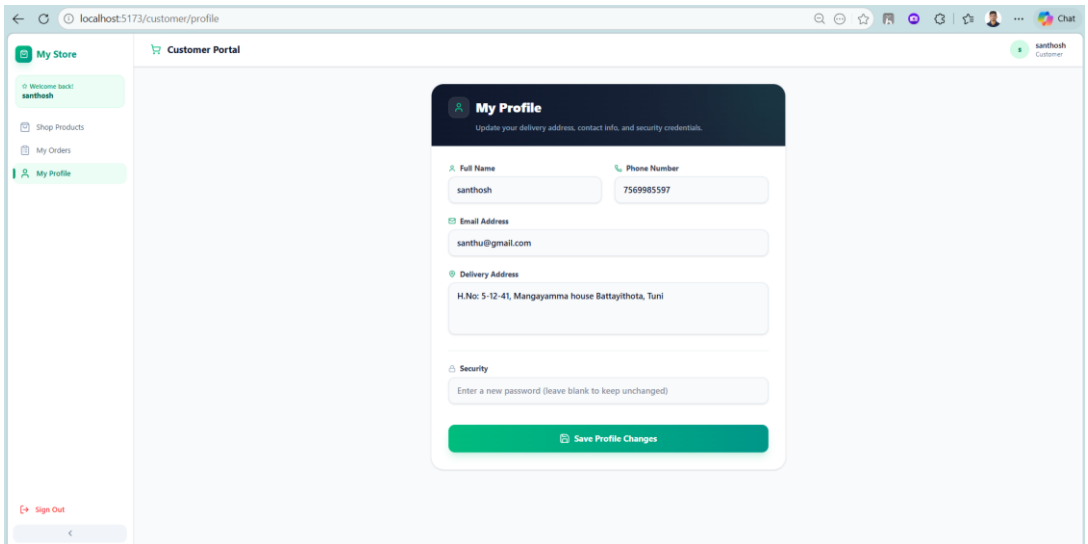
Payment Successful:



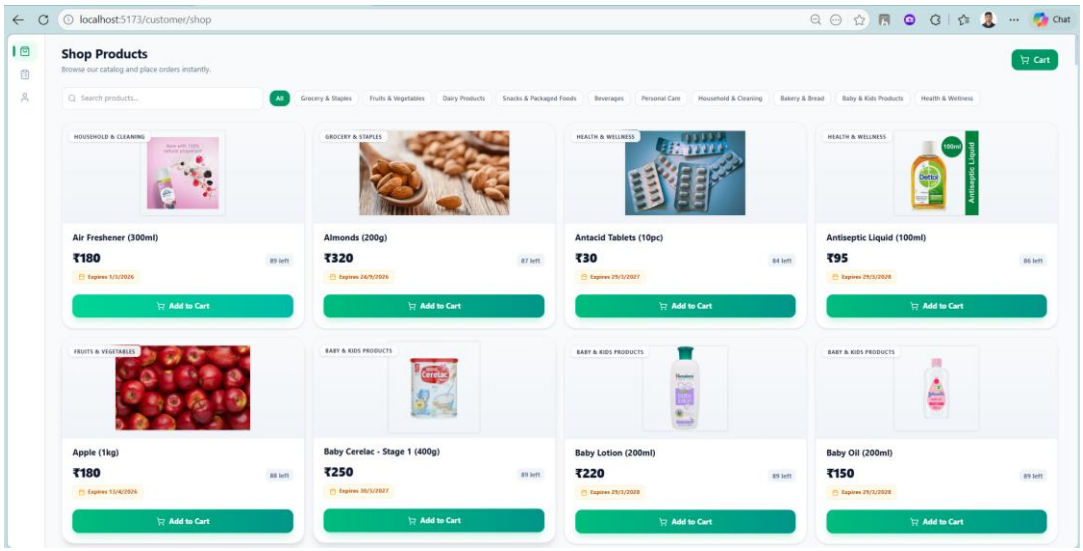
My orders:



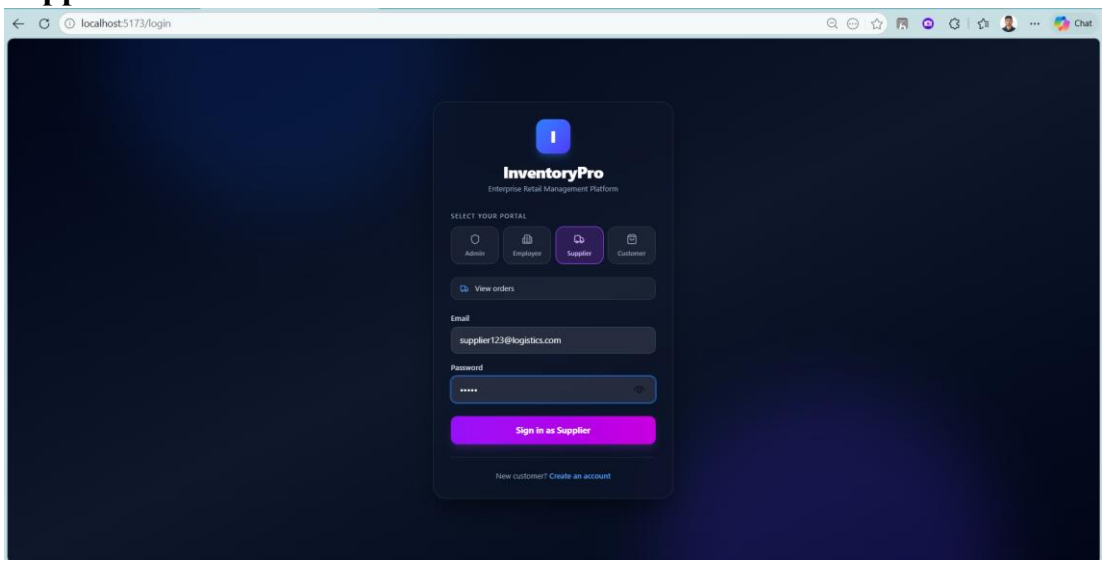
Update Profile:



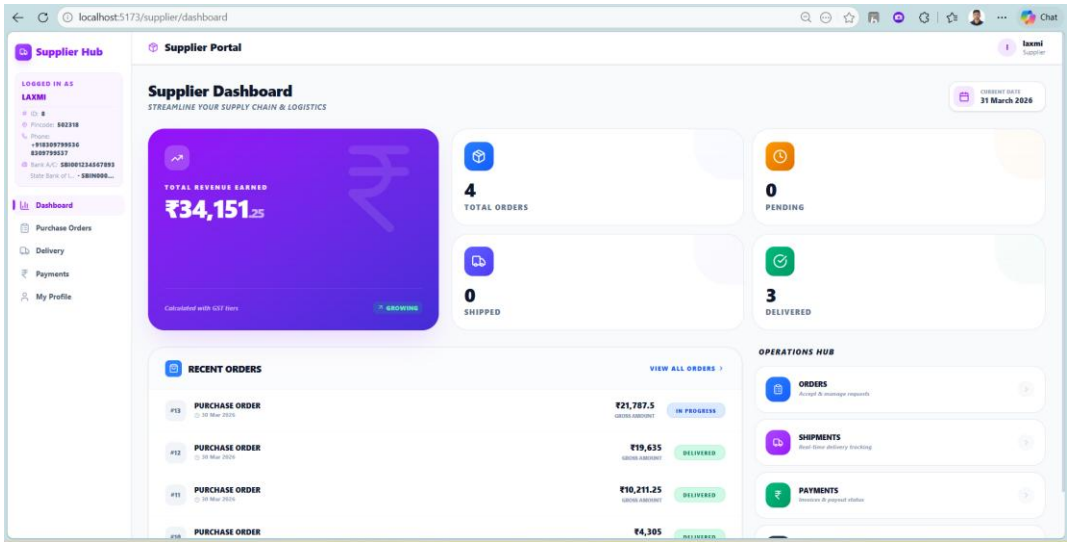
Products in store:



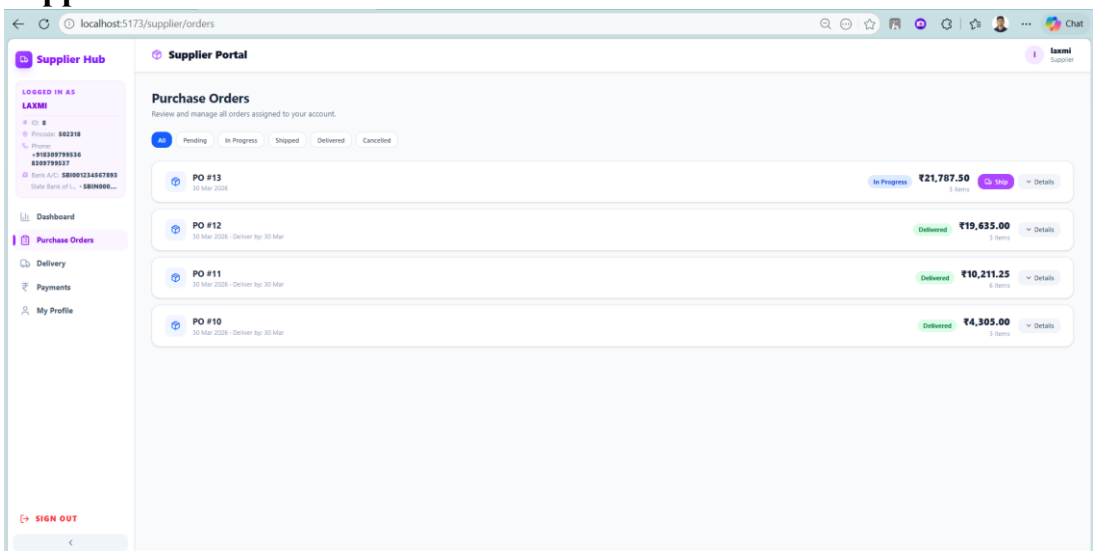
Supplier Portal:



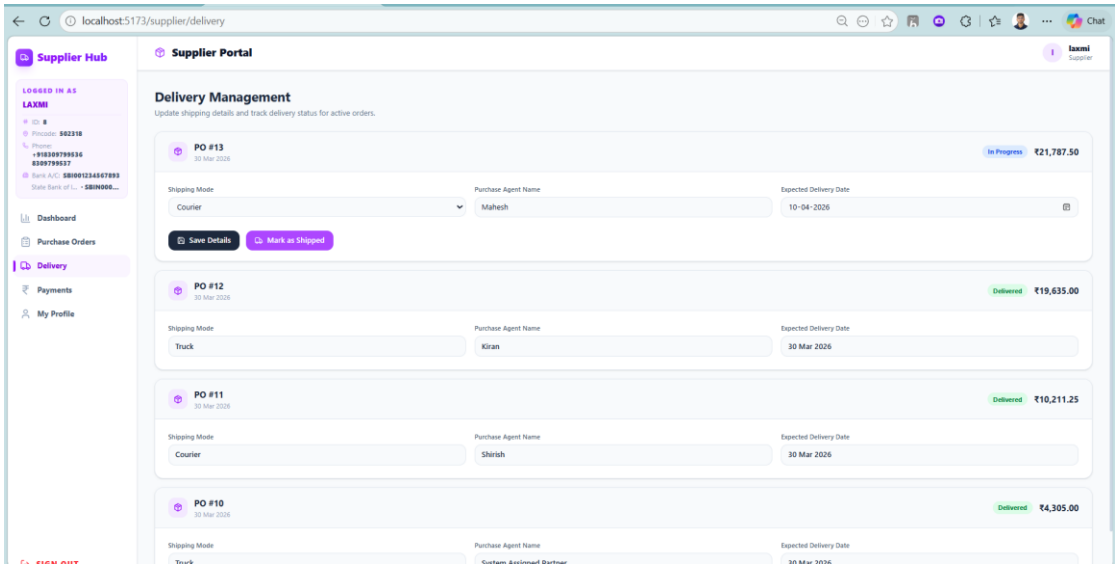
Supplier Dashboard:



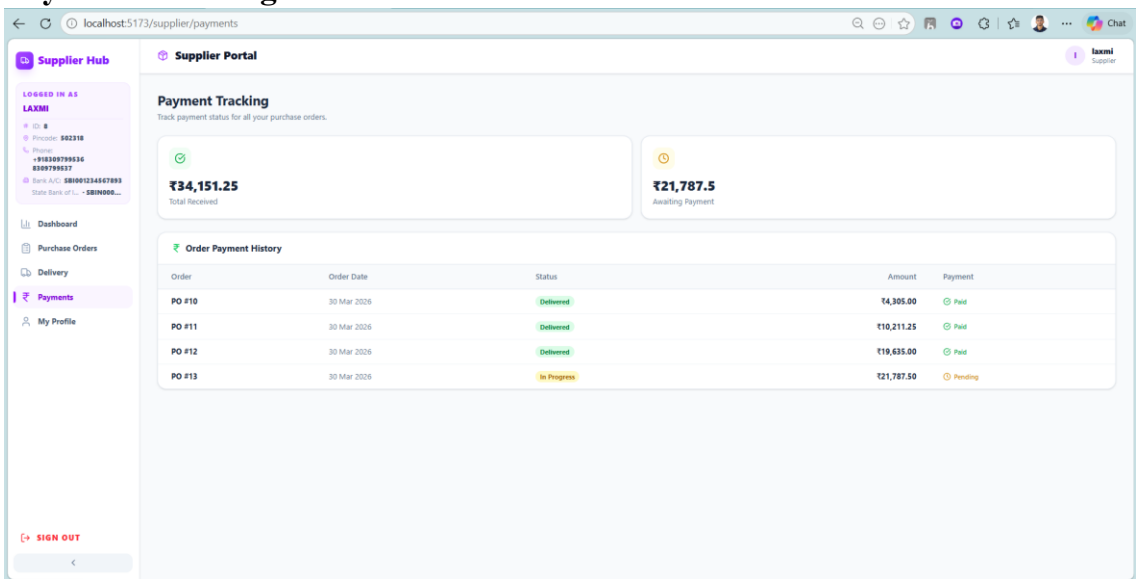
Supplier Orders:



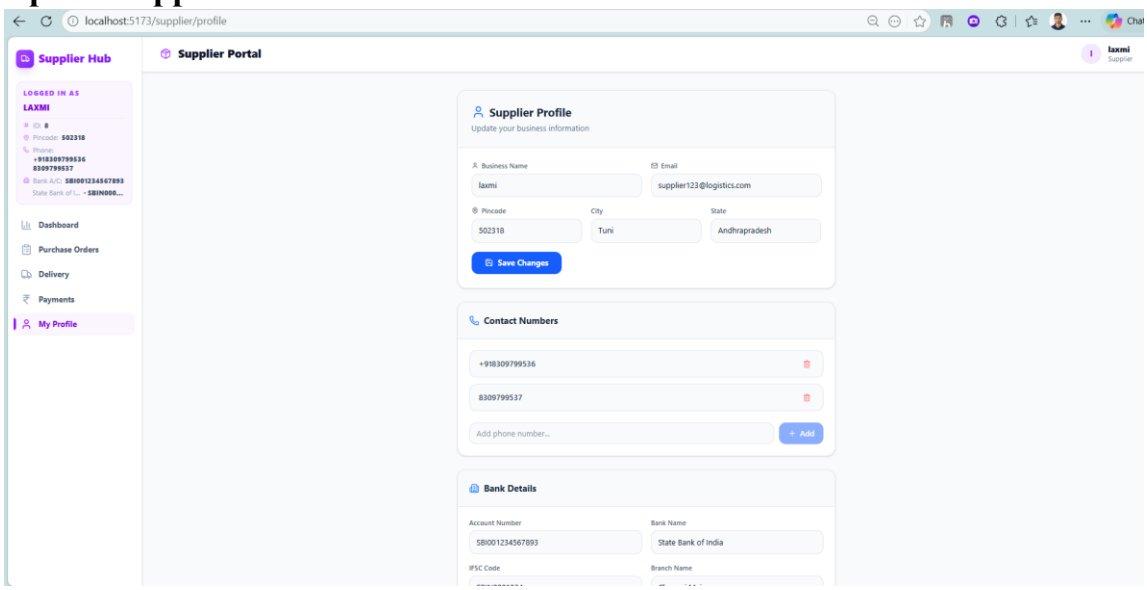
Supplier Delivery Details:

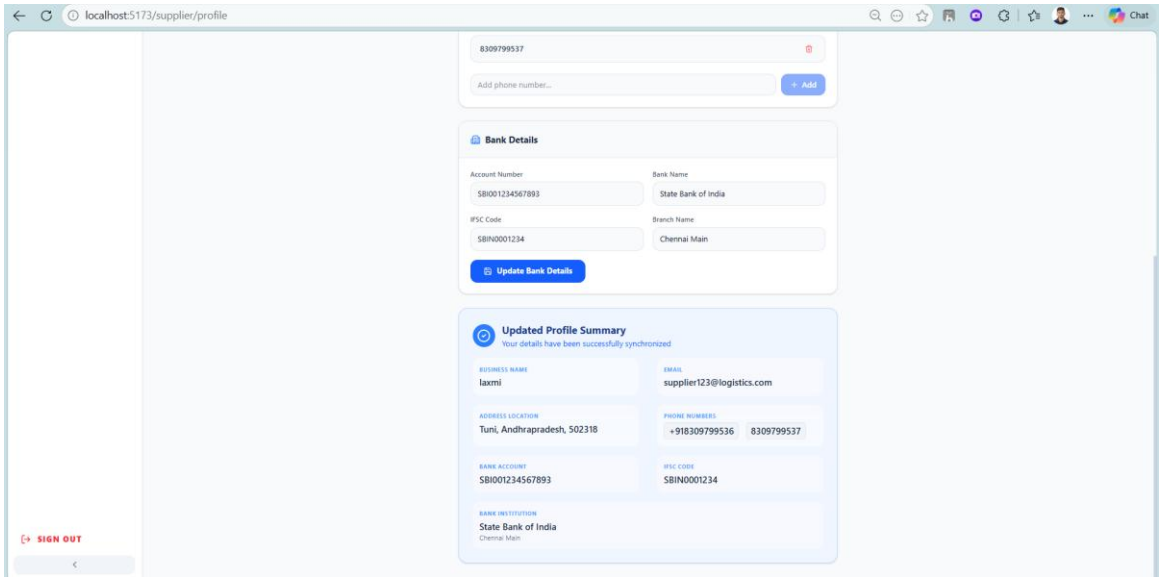


Payment Tracking:

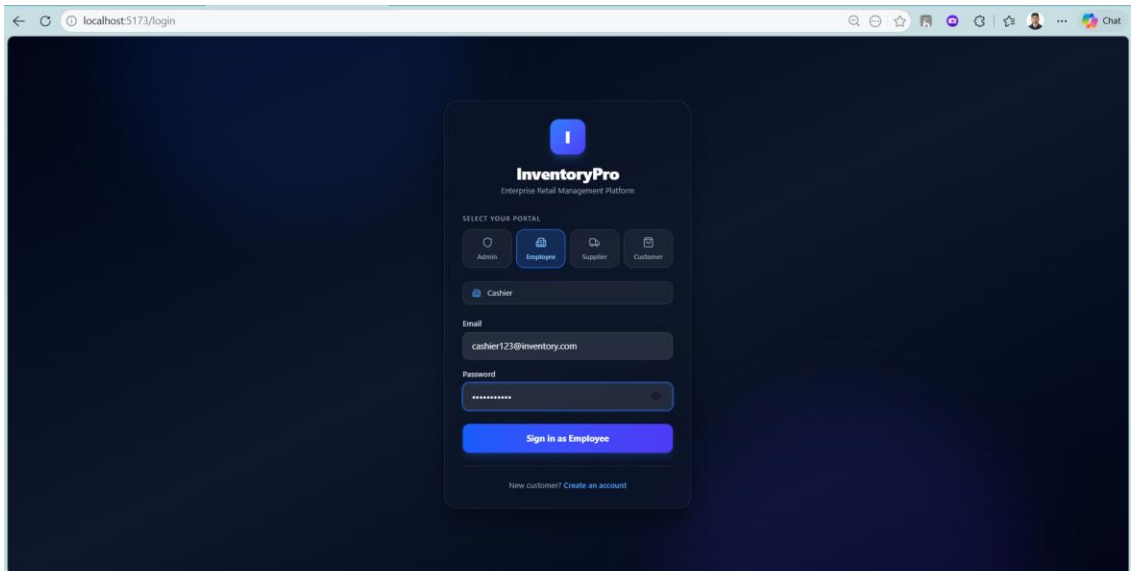


Update supplier Profile:

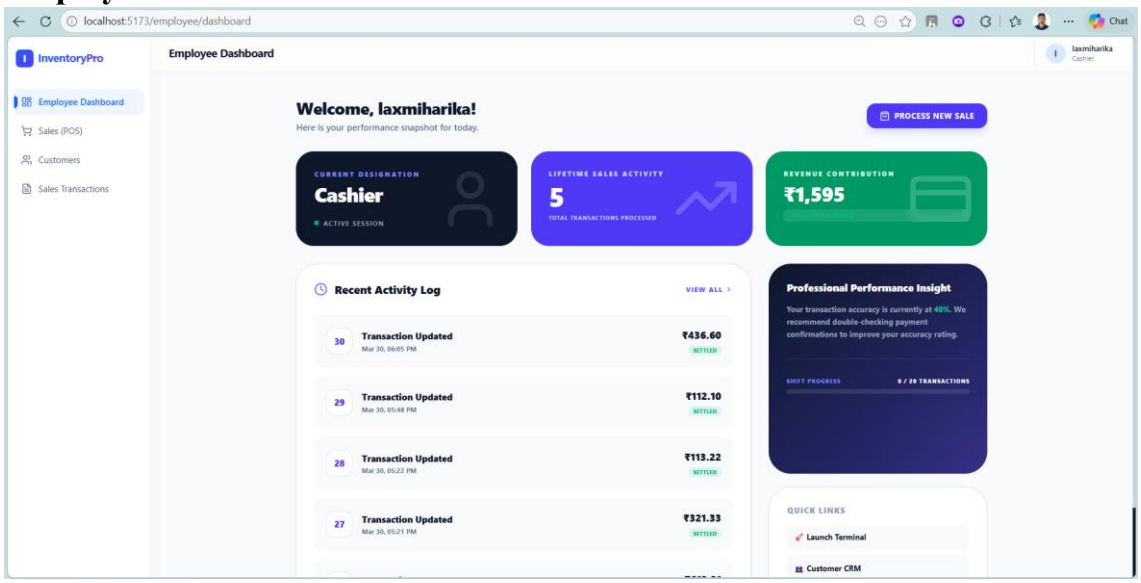




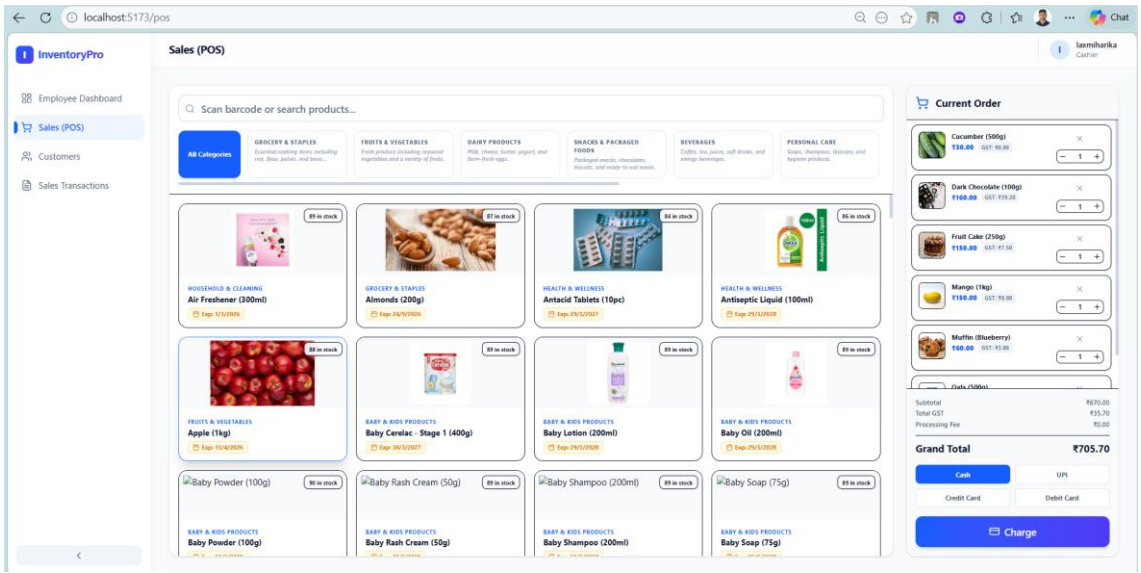
Employee Portal:

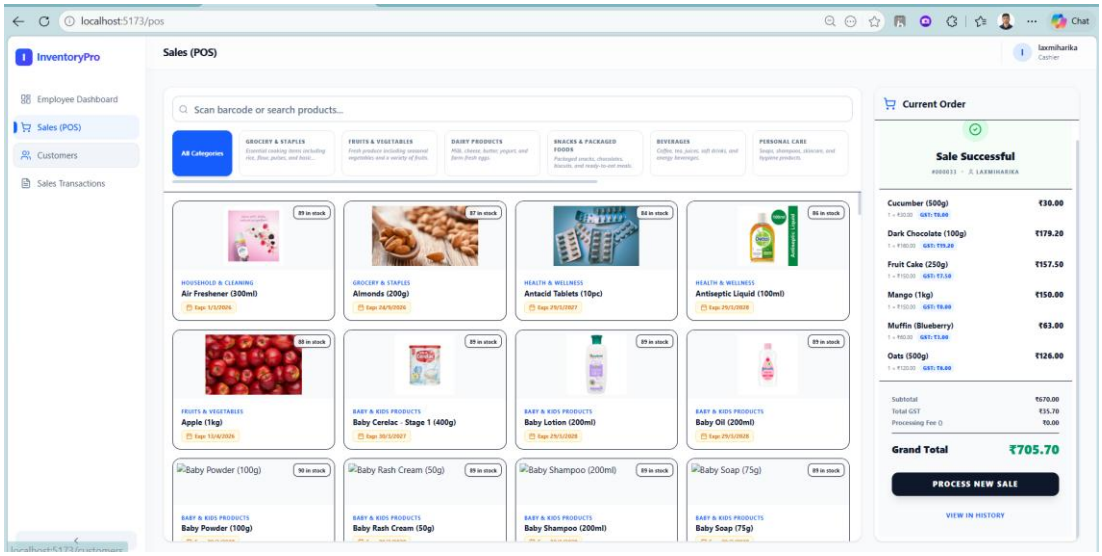


Employee Dashboard:

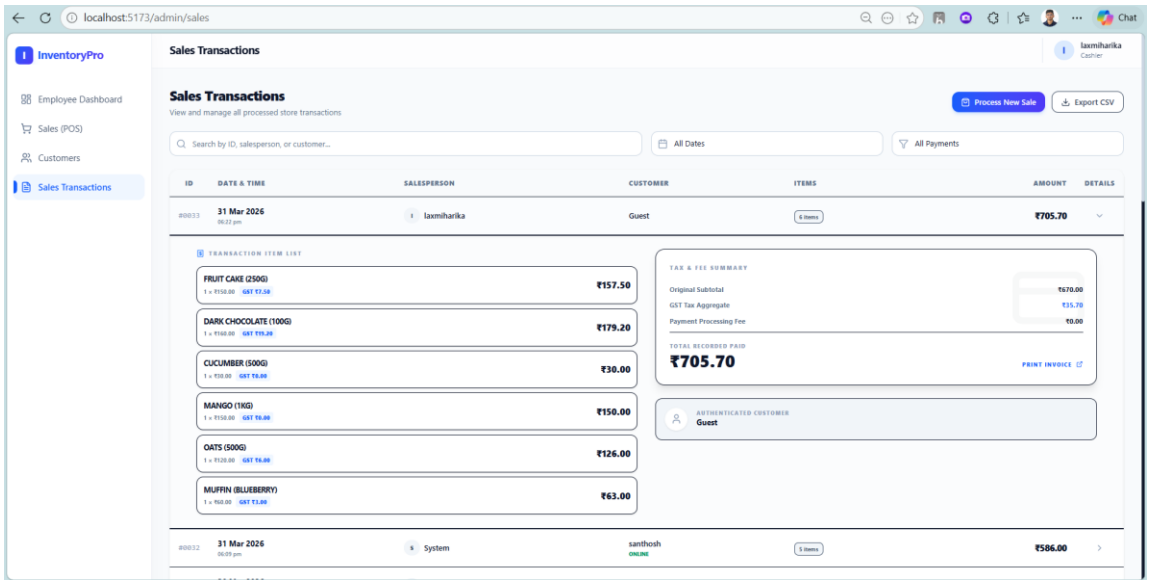


Process New Sale:

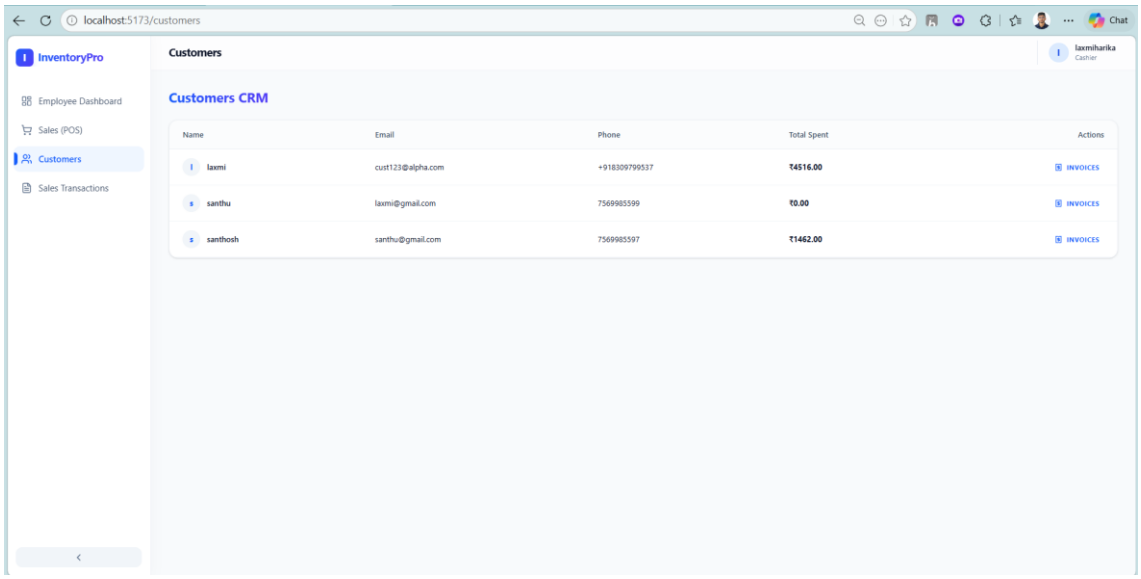




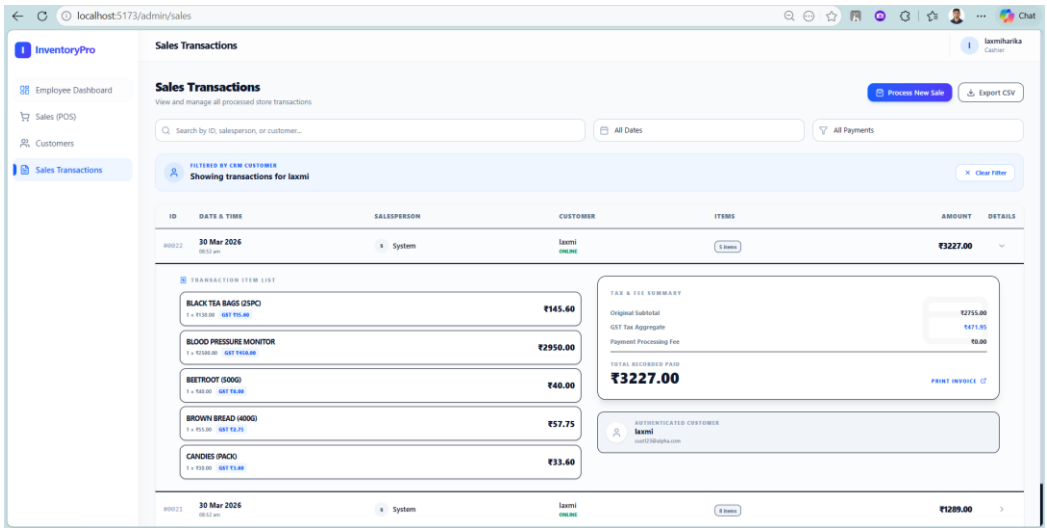
Sales Transaction Histroy:



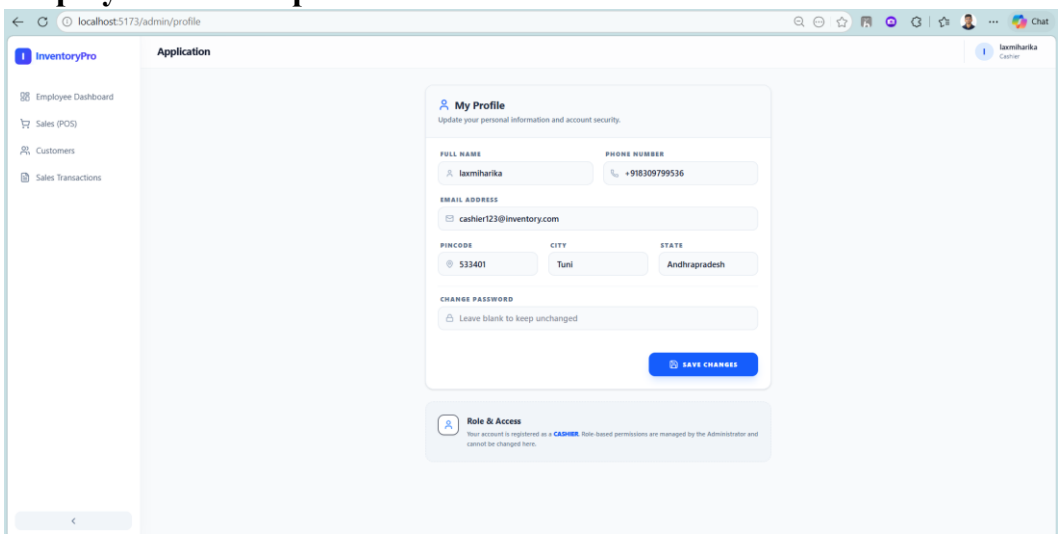
Online Customers:



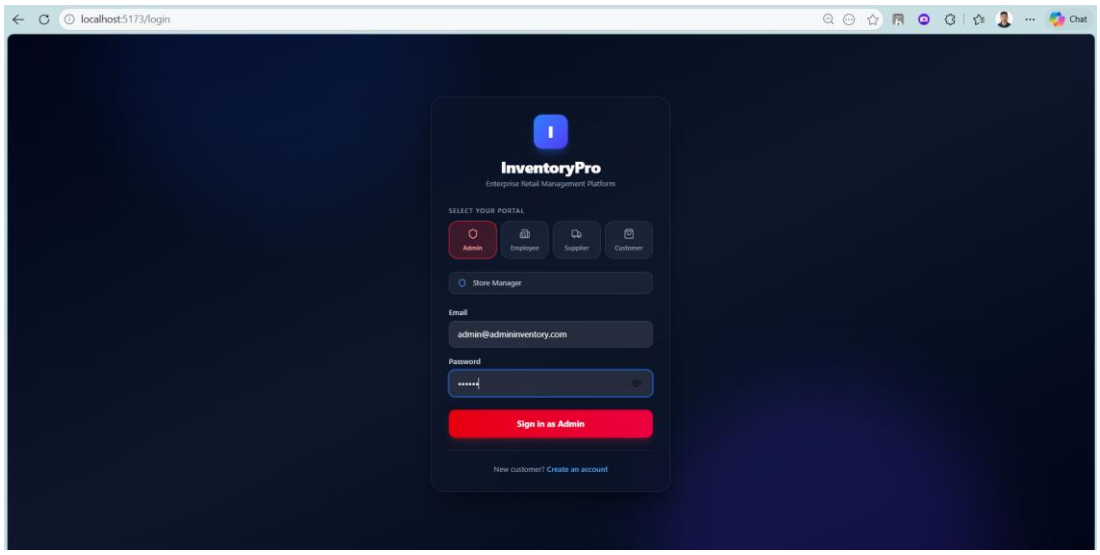
Invoices Generated by online Customers:



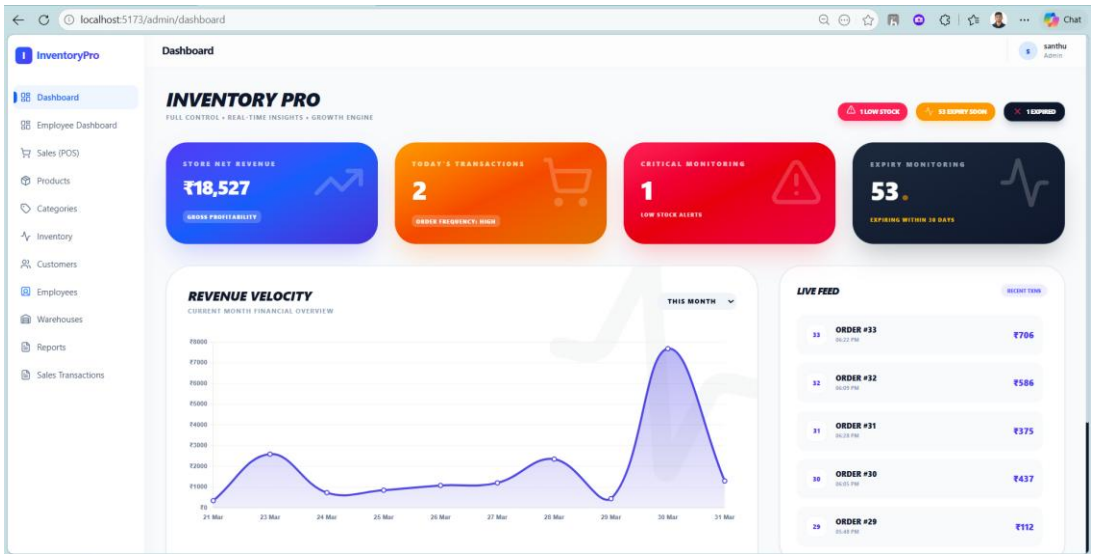
Employee Profile Update:

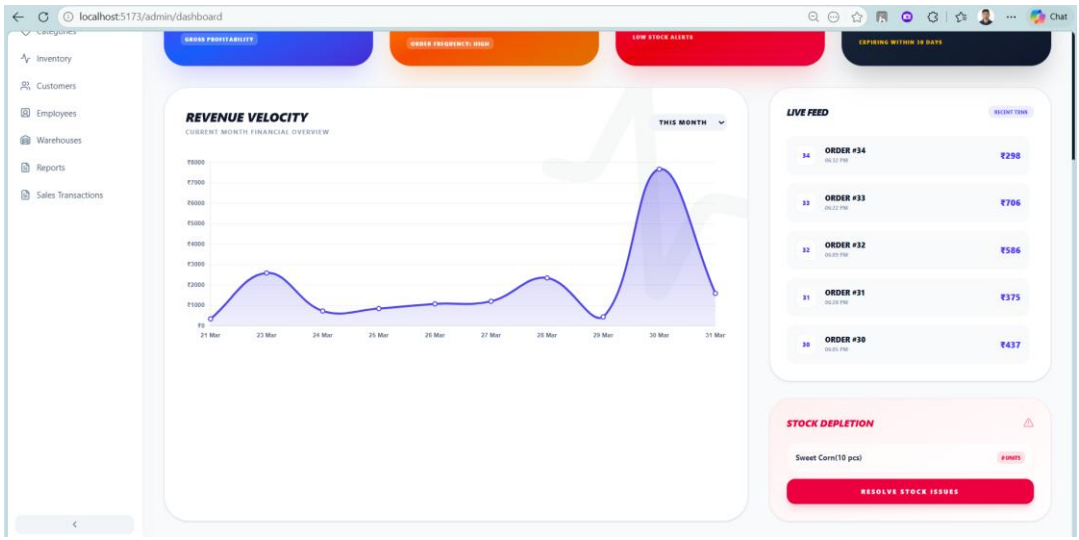


Admin Portal:

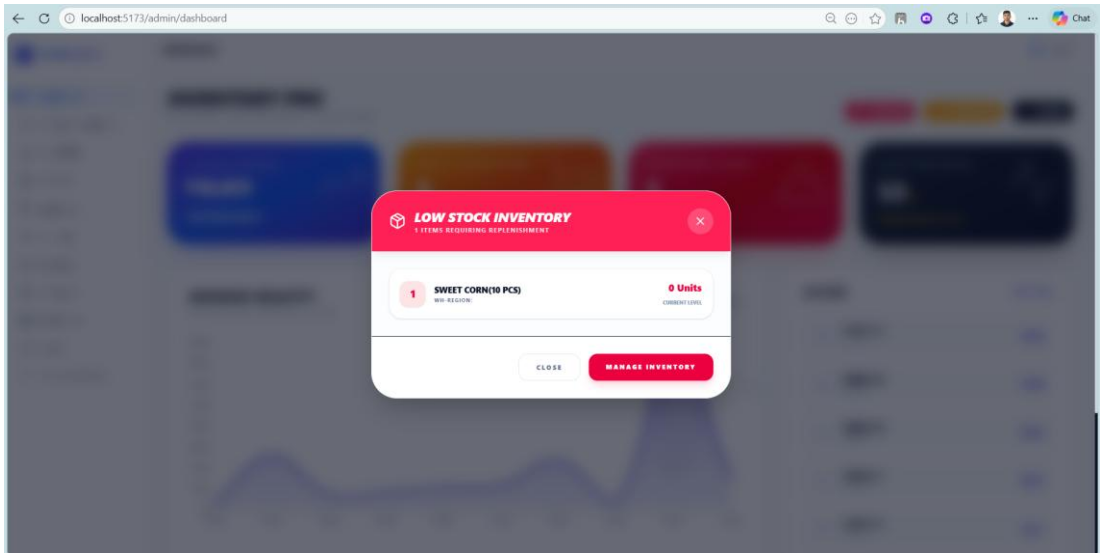


Admin Dashboard:

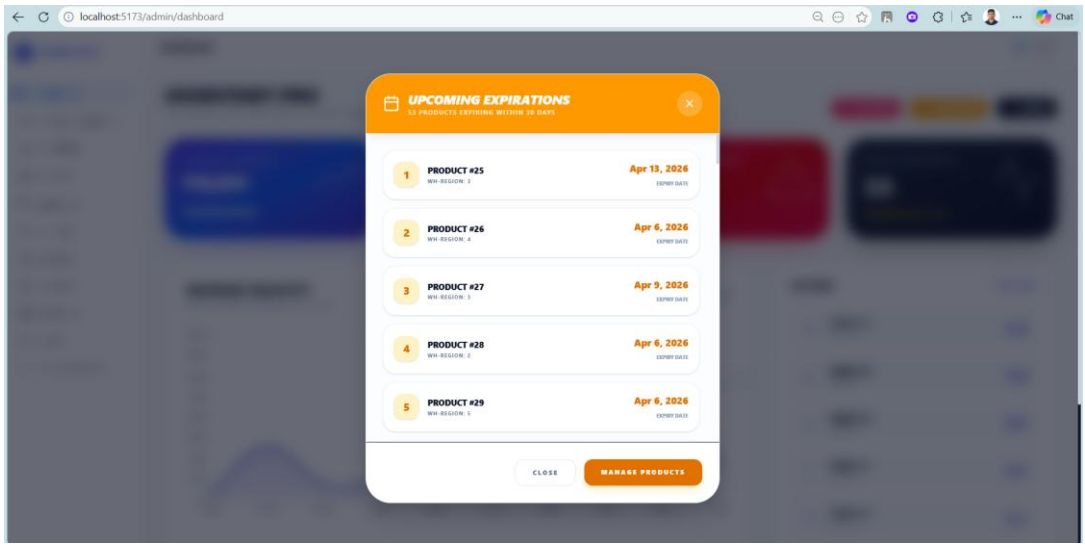




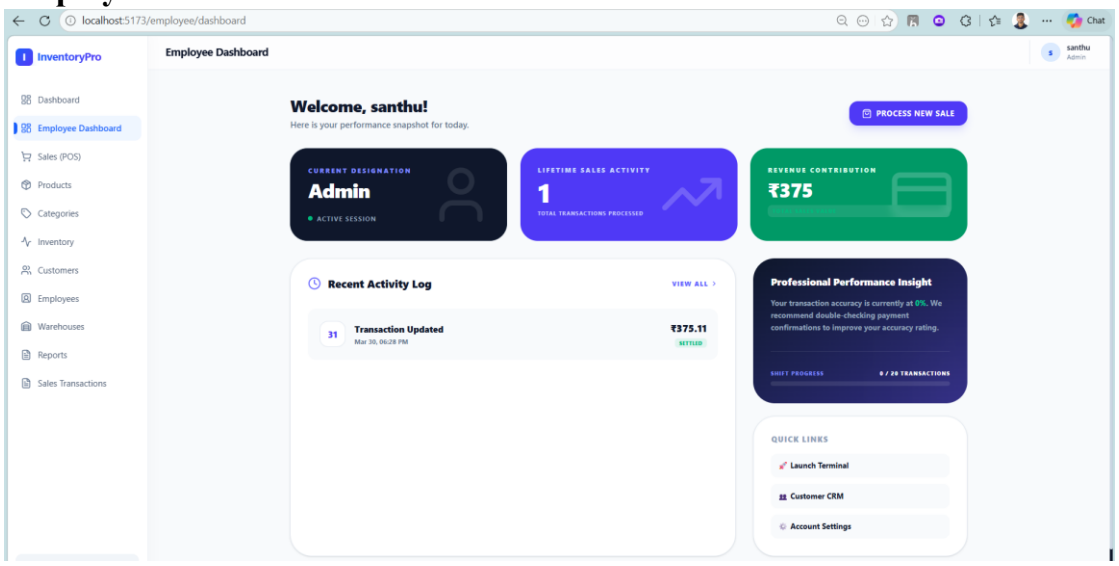
Low Stock Alert:



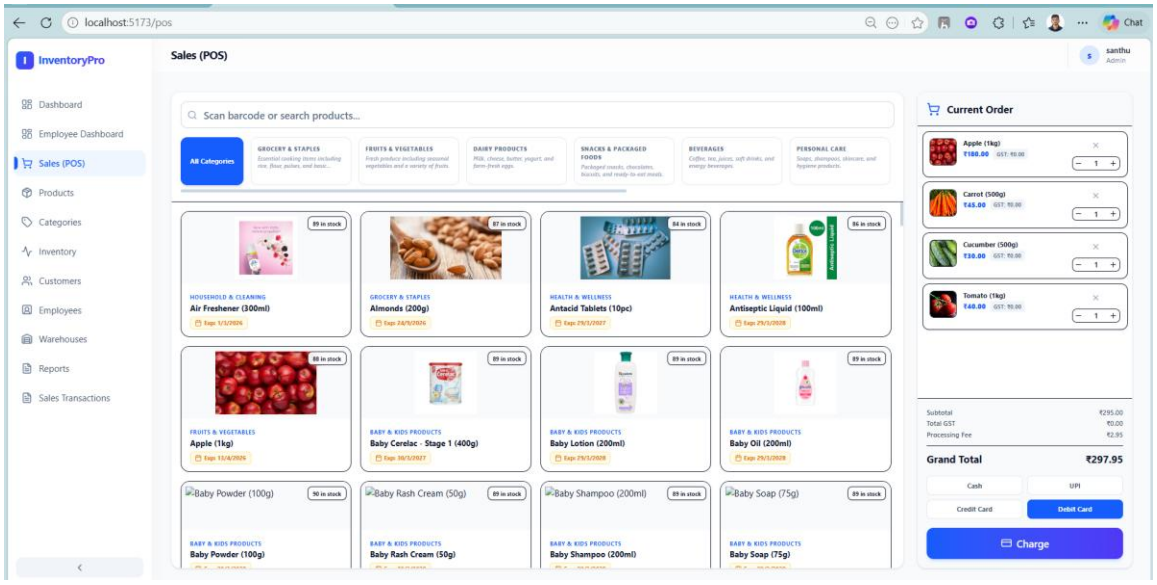
Expiry Alerts:

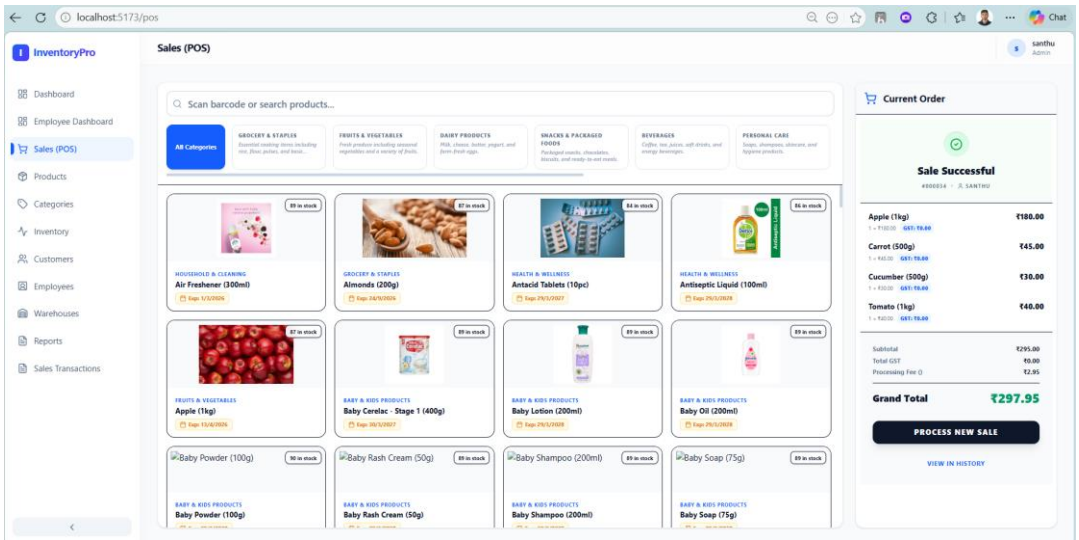


Employee Dashboard:

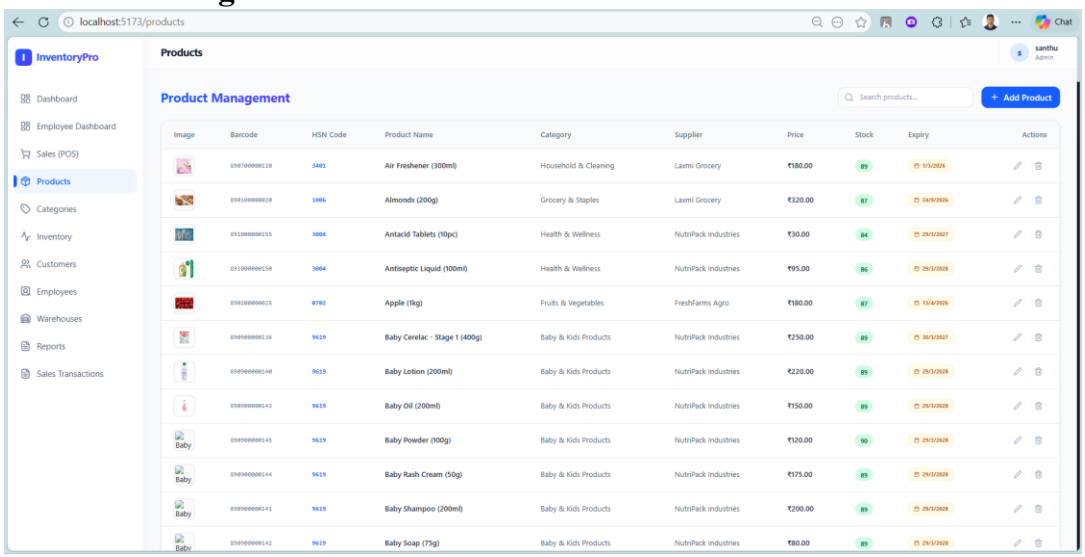


Admin sales Transaction:

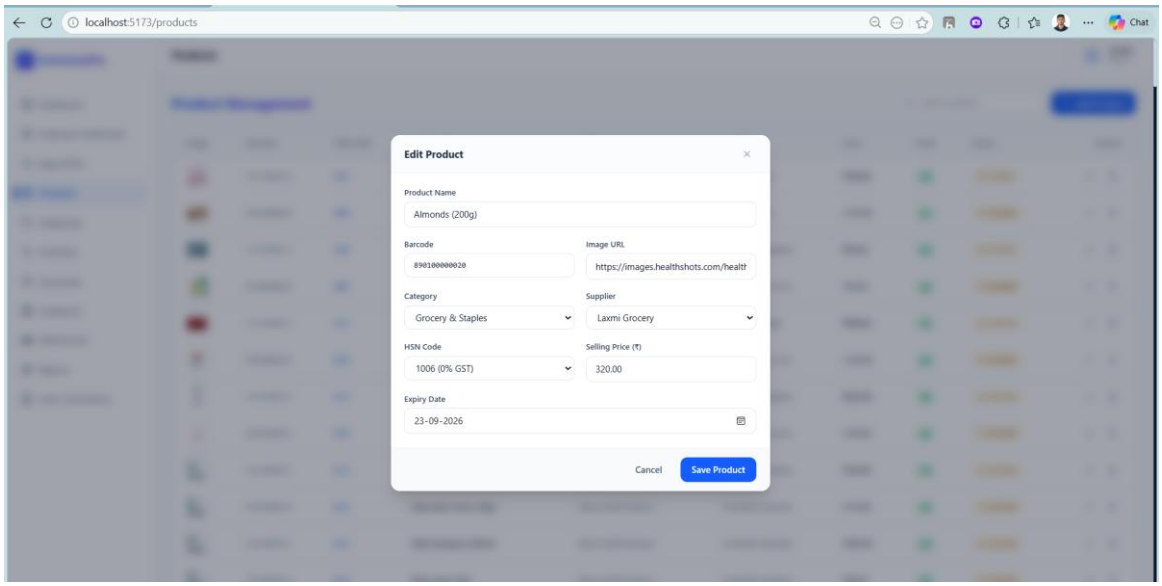




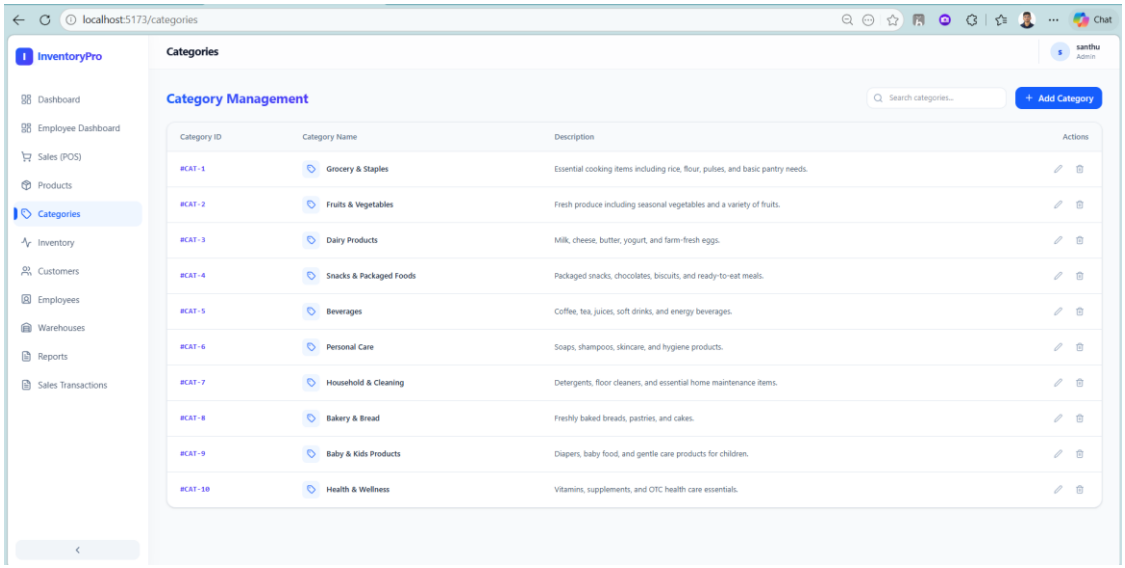
Product Management:



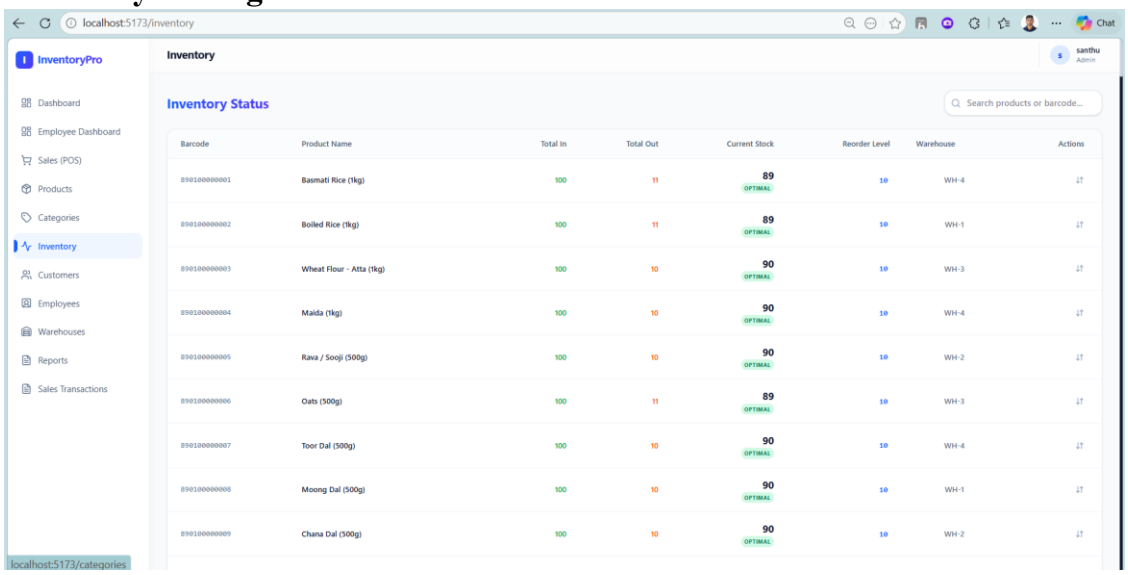
Edit Product:



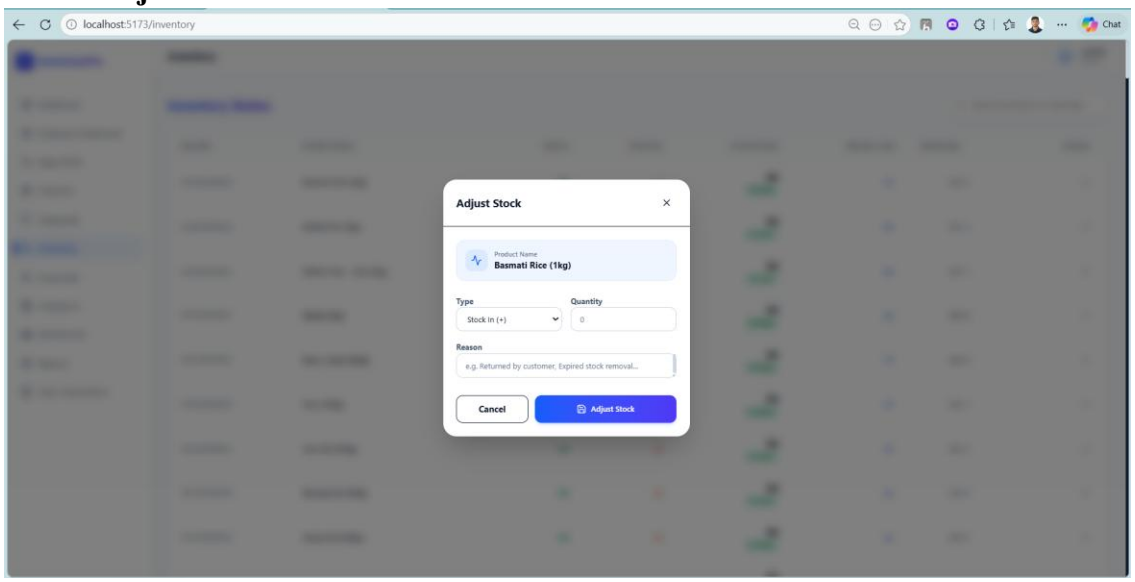
Category Management:



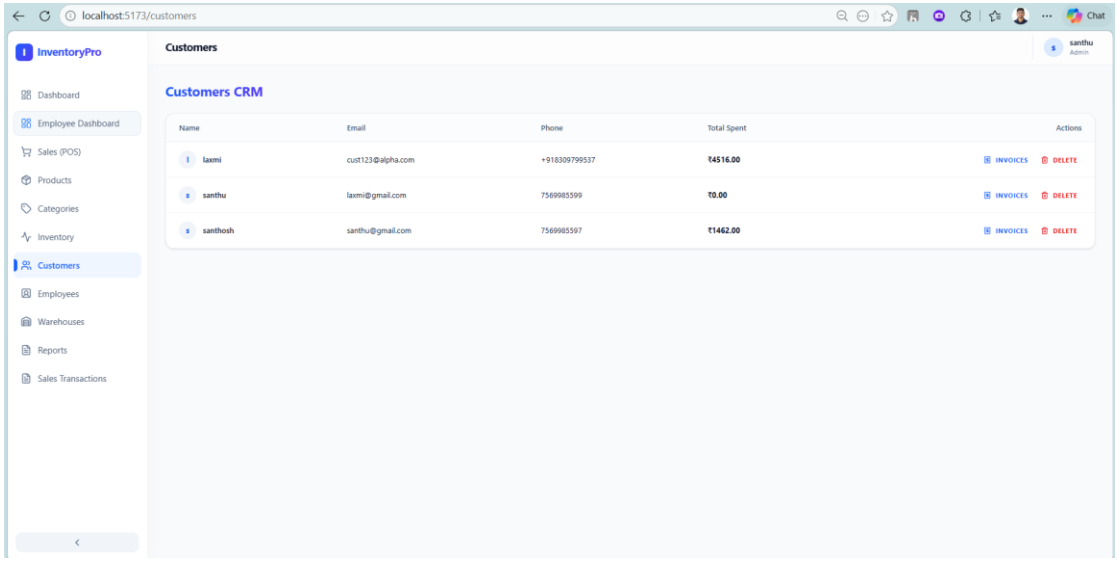
Inventory Management:



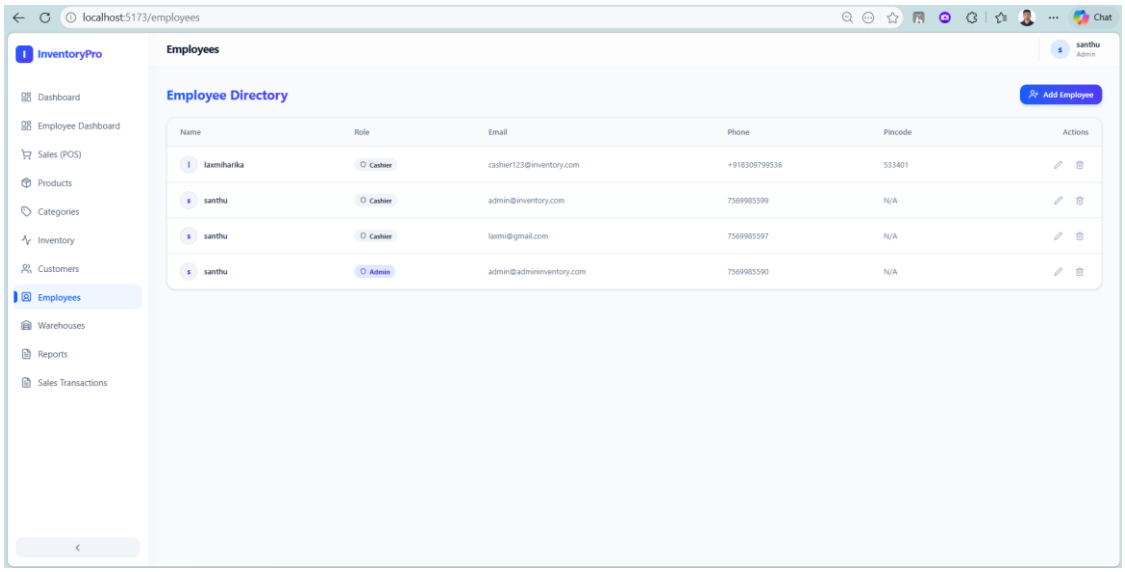
Stock Adjustment:



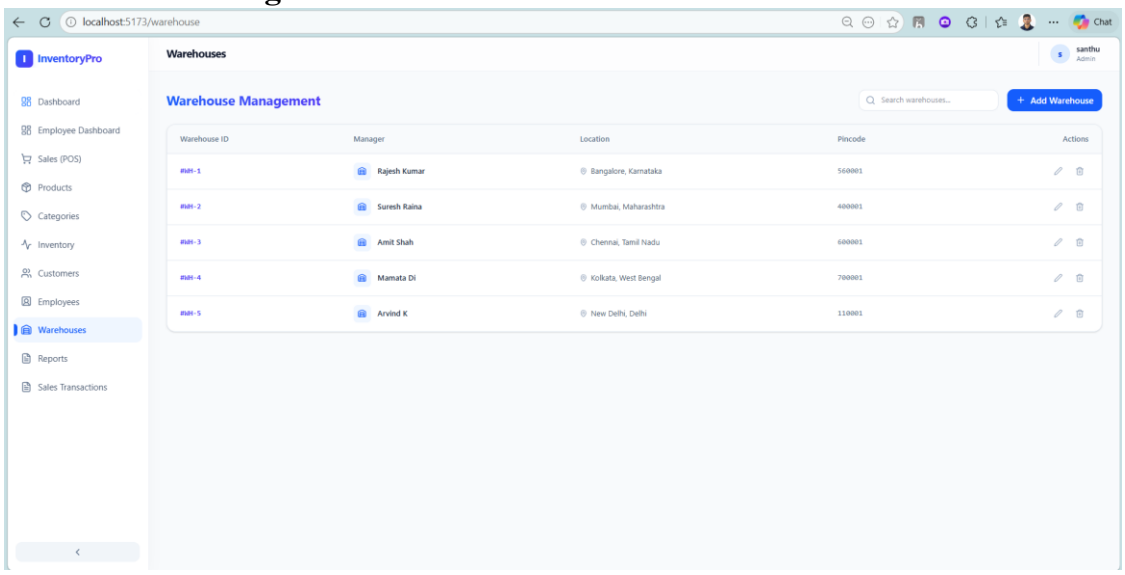
Online Customers:



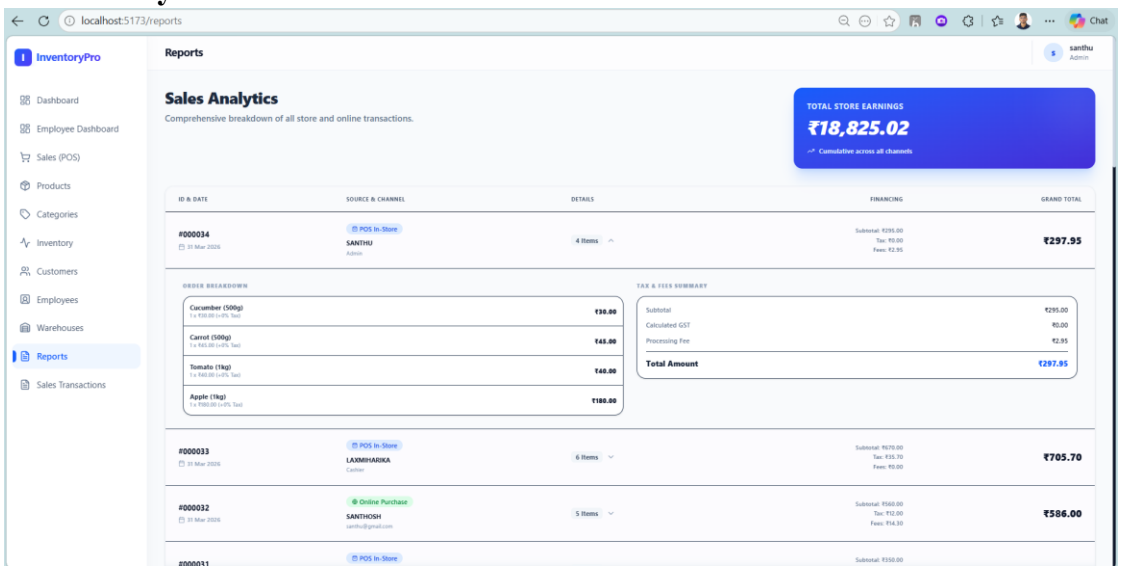
Employees:



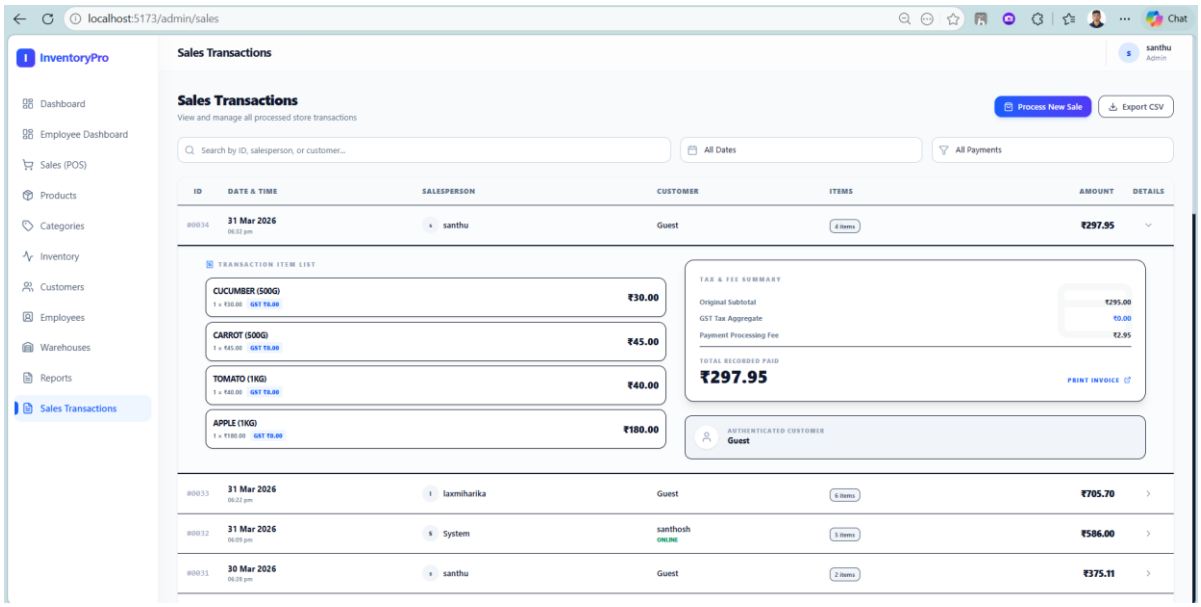
Warehouse Management:



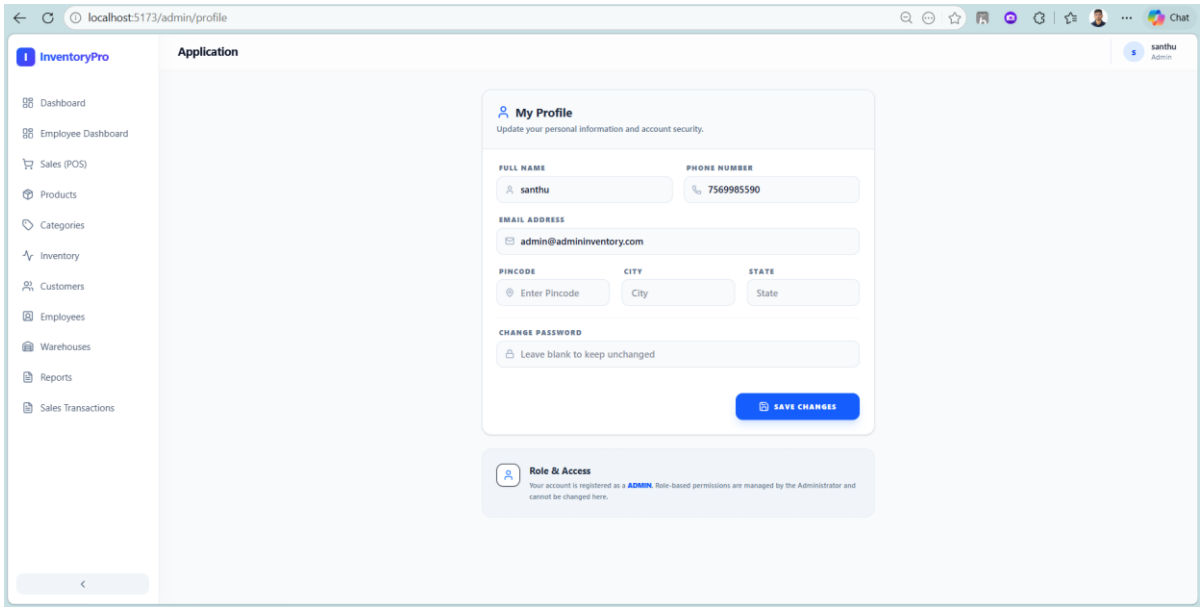
Sales Analytics:



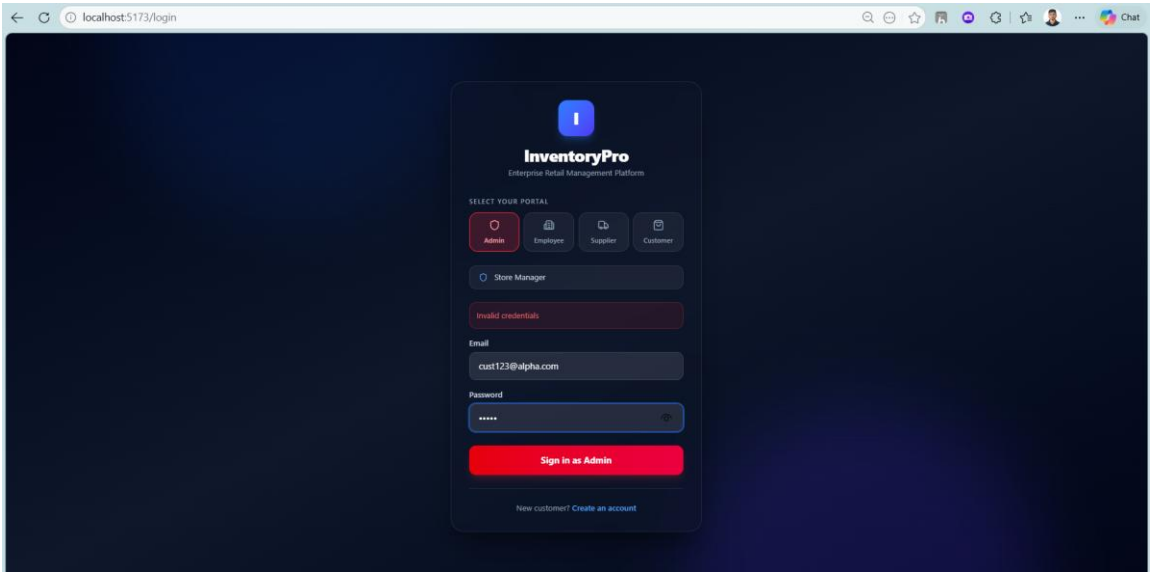
Sales Tranactions:



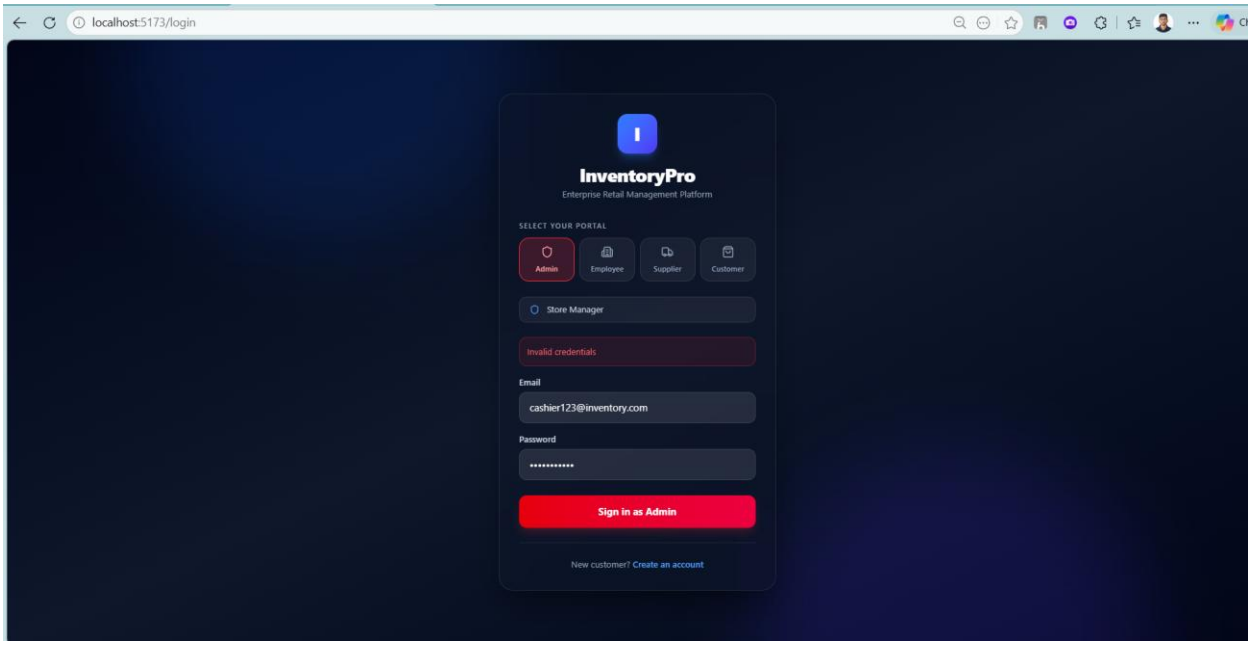
Profile update:



Protected Routes:
Customer cannot access admin portal:



Employee cannot have access to admin portal:



Users are restricted to authentication through their respective portals based on their roles.

CONCLUSION

InventoryPro successfully demonstrates the design, development, and deployment of a comprehensive inventory and sales management system built on strong database principles and modern web technologies. The system addresses the critical limitations of existing commercial tools by providing an open, normalised relational schema, genuine role-differentiated multi-portal access, automated real-time stock and expiry alerting, and an integrated Point-of-Sale module with HSN/GST compliance — all within a single, cohesive web application.

The system's frontend, built with React.js, Framer Motion, and TanStack Query, delivers a premium user experience characterised by real-time data synchronisation, smooth micro-animations, and an intuitive interface that reduces the training burden for retail staff. The backend, built on Node.js and Express.js with a MySQL connection pool, provides a stateless, scalable REST API secured by JWT authentication. The database schema, normalised to ***Boyce-Codd Normal Form (BCNF)*** across 25tables, ensures data integrity through a comprehensive network of primary keys, foreign key constraints and unique indices.

From a DBMS perspective, this project validates the practical utility of relational database theory - including normalisation, constraint enforcement, and complex multi-table JOIN queries - in solving real-world business problems. The expiry monitoring feature, for example, is entirely driven by date arithmetic in SQL, demonstrating that sophisticated business logic can be implemented directly within the data tier.

The current implementation of InventoryPro offers a strong foundation for inventory management and sales management, but there are a number of enhancements that can be made in future versions to take this system to a higher level of functionality, integration, and scalability. One of the key enhancements integrating the Admin and Supplier modules. Currently, both portals operate separately, so admins cannot directly place purchase orders with suppliers. In future, this can be improved by enabling a unified system for creating, tracking, and managing purchase orders. Another key enhancement would be to add support for a mobile application, which would enable users to use this system on-the-go, along with a barcode/QR code scanner for enhanced accuracy in billing and inventory management. Facilities such as AI-based demand forecasting can be utilized to forecast stock requirements. Integration with email and SMS notifications can be done to improve real-time communication. Use of cloud infrastructure for deploying the system can be done to improve availability and scalability. Improving security mechanisms such as two-factor authentication can be done to improve data protection. Use of advanced analytics and reporting tools can be done to improve business insights, thus facilitating decision-making and improving efficiency.

REFERENCES

1. R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.

2. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

3. S. B. Navathe and R. Elmasri, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson Education, 2016.

4. D. Flanagan, JavaScript: The Definitive Guide, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

5. Oracle Corporation, MySQL 8.0 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

6. Meta Platforms, React Documentation. [Online]. Available: https://react.dev/

7. Express.js Documentation. [Online]. Available: https://expressjs.com/

8. TanStack Query v5 Documentation. [Online]. Available: https://tanstack.com/query/latest

9. Framer Motion Documentation. [Online]. Available: https://www.framer.com/motion/

10. Tailwind Labs, Tailwind CSS Documentation. [Online]. Available: https://tailwindcss.com/docs

11. Auth0, JSON Web Token Introduction. [Online]. Available: https://jwt.io/introduction

12. OpenJS Foundation, Node.js Documentation. [Online]. Available: https://nodejs.org/docs/

13. E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

14. P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston, MA, USA: Addison-Wesley, 2013.

15. R. J. Wieringa, Design Science Methodology for Information Systems and Software Engineering. Berlin, Germany: Springer, 2014.

COMPLETE IMPLEMENTATION

Due to the volume of code, it is organised by module below
Full source files are available in the project repository.
https://github.com/Laxmiharika522/Design-and-Implementation-of-an-Inventory-and-Sales-Management-System-for-a-Retail-Store

Backend (Node.js / Express.js)
The server src/ directory contains:

- **index.js** — Express application bootstrap, middleware registration, route mounting, and server startup with database connection pooling.
- **config/db.js** — MySQL2 connection pool configuration.
- **middleware/authMiddleware.js** — JWT-based authentication including token verification and role-based authorization.
- **controllers/** — Contains business logic for all modules such as reportController, inventoryController, productController, supplierPortalController, and customerPortalController.
- **routes/** — Contains 14 route modules that map HTTP requests (GET, POST, PUT, DELETE) to corresponding controller functions.
- **models/** — Contains SQL schema definitions and database structure files used for table creation and setup.
- **utils/** — Contains helper functions used across modules for calculations, formatting, and standard response handling.

Frontend (React.js)
The client **src/** directory contains:

- **main.jsx** — Application entry point; initializes React and renders the root component.
- **App.jsx** — Main routing configuration with role-based access control and protected routes.
- **context/AuthContext.jsx** — Manages global authentication state, user roles, and session persistence.
- **layouts/** — Contains layout components for different portals (DashboardLayout, SupplierLayout, CustomerLayout) with navigation and UI structure.
- **pages/** — Contains all UI modules organized by role (admin, employee, supplier, customer) including dashboards, POS, inventory, products, orders, and profile management.
- **index.css / App.css** — Global styling using Tailwind CSS along with custom styles and animations.